Mechanics of Materials 205 (2025) 105317

Contents lists available at ScienceDirect

Mechanics of Materials

journal homepage: www.elsevier.com/locate/mecmat

Research paper

On the application of Physically-Guided Neural Networks with Internal Variables to Continuum Problems

Rubén Muñoz-Sierra [a,b], Jacobo Ayensa-Jiménez [a,b], Manuel Doblaré [a,b,c,d,e,*]

[a] TME Lab, Aragón Institute of Engineering Research (I3A), University of Zaragoza, Mariano Esquillor, S/N, 50018 Zaragoza, Spain
[b] Institute for Health Research Aragon (IISA), Avenida San Juan Bosco, 13, 50009 Zaragoza, Spain
[c] Mechanical Engineering Department, School of Engineering and Architecture (EINA), University of Zaragoza, María de Luna, 3, 50018 Zaragoza, Spain
[d] CIBER-BBN, ISCIII, Avenida Monforte de Lemos, 3, 28029, Madrid, Spain
[e] School of Material Science and Engineering, Nanjing Tech University, Hongjing Avenue, Jiangning District, 211167, Nanjing, China

ARTICLE INFO

Dataset link: https://github.com/rmunozTMELab/PGNNIV-to-Continuum-Problems.git

Keywords:
Physically Guided Neural Networks with Internal Variables
Explanatory Artificial Intelligence
Scientific machine learning
Internal state variables
Continuum physics

ABSTRACT

Predictive physics has been historically based upon the development of mathematical models that describe the evolution of a system under certain external stimuli and constraints. The structure of such mathematical models relies on a set of physical hypotheses that are assumed to be fulfilled by the system within a certain range of environmental conditions. A new perspective is now raising that uses physical knowledge to inform the data prediction capability of Machine Learning tools, coined as Scientific Machine Learning.

A particular approach in this context is the use of Physically-Guided Neural Networks with Internal Variables, where universal physical laws are used as constraints to a given neural network, in such a way that some neuron values can be interpreted as internal state variables of the system. This endows the network with unraveling capacity, as well as better predictive properties such as faster convergence, fewer data needs and additional noise filtering. Besides, only observable data are used to train the network, and the internal state equations may be extracted as a result of the training process, so there is no need to make explicit the particular structure of the internal state model, while getting solutions consistent with Physics.

We extend here this methodology to continuum physical problems driven by a general set of partial differential equations, showing again its predictive and explanatory capacities when only using measurable values in the training set. Moreover, we show that the mathematical operators developed for image analysis in deep learning approaches can be used in a natural way and extended to consider standard functional operators in continuum Physics, thus establishing a common framework for both.

The methodology presented demonstrates its ability to discover the internal constitutive state equation for some problems, including heterogeneous, anisotropic and nonlinear features, while maintaining its predictive ability for the whole dataset coverage, with the cost of a single evaluation. As a consequence, microstructural material properties can be inferred from macroscopic measurement coming from sensors without the need of specific homogeneous test plans neither specimen extraction.

1. Introduction

In the last years, our capacity to collect data has increased at an unprecedented rate (Manyika et al., 2011). Today, billions of sensors, transducers or videocameras get data from physical systems, while the advent of the Internet of Things (IoT) will hugely increase the amount and variety of such data (Atzori et al., 2010). Besides, the current technology allows to improve the predictive capability of physical models by adding information from the experimental data, something known as Dynamic Data-Driven Systems (DDDS) (Darema, 2004; Peherstorfer and Willcox, 2015; Kutz et al., 2016; Ayensa-Jiménez et al., 2018). This ability is progressively moving us from the parametric regression approach used in model-based science to non-parametric regression methods, typical of Artificial Intelligence (AI) and, in particular, of Artificial Neural Networks (ANN) (Maggiora et al., 1992), a technique that has demonstrated an impressive success in many fields (Stulp and Sigaud, 2015; Hoffmann et al., 2019; Aneshensel, 2013; Raghupathi and Viju, 2014; Lopez-Moreno et al., 2014; Krizhevsky et al., 2012; Purwins et al., 2019). This new paradigm may be interpreted as a certain return from the well-established hybrid inductive-deductive approach, to the original pure inductive method that only uses direct regressions

* Corresponding author at: TME Lab, Aragón Institute of Engineering Research (I3A), University of Zaragoza, Mariano Esquillor, S/N, 50018 Zaragoza, Spain.
E-mail addresses: 739163@unizar.es (R. Muñoz-Sierra), jacoboaj@unizar.es (J. Ayensa-Jiménez), mdoblare@unizar.es (M. Doblaré).

https://doi.org/10.1016/j.mechmat.2025.105317
Received 4 September 2024; Received in revised form 27 February 2025; Accepted 3 March 2025
Available online 17 March 2025

between input and output, without imposing any additional hypothesis on the mathematical structure of the regression model (Minto, 2018). This move back is being helped by the huge amount and variety of the data available, the power of data augmentation and preprocessing techniques (Maharana et al., 2022), especially for non-structured data, the increasing computer power with dedicated capacities (LeCun, 2019) and a new generation of software tools that simplify and optimize the construction of the networks, the training process, and the validation of ANN-based models, such as TensorFlow and Keras (Gulli and Pal, 2017), Theano (Bergstra et al., 2011) or Pytorch (Paszke et al., 2019).

This latter approach still faces some important drawbacks in its application to physical sciences. Scientific data are biased by centuries of knowledge (Berry, 2011; Gould, 1981). In addition, spurious correlations between them may be unnoticed by AI methods (Kitchin, 2013, 2014), so a blind algorithm without any additional information may lead to wrong predictions. Also, many times, we lack sufficient data in terms of quantity, variety, and quality to extract the characteristic features in problems that often involve a large number of variables that interact in a complex manner, even if some recently proposed mitigation strategies are used (Chai et al., 2024). As a consequence, we can expect poor extrapolation capacity of such models or, alternatively, overfitting behavior (Xue, 2019). Finally, a physically-based model is also useful to get new information by interpreting its structure, parameters, and mathematical properties, being this the reason for the important efforts made to whitening the black-box way of working of current machine-learning predictive algorithms (Xu et al., 2019), making important the difference between information and knowledge (Kettinger and Li, 2010). All these characteristics, with especial emphasis on the latter, have delayed, in Physics and Engineering, the success that data-driven applications have achieved in other domains. Despite these limitations, data-based models, helped by AI and in particular Machine Learning (ML) techniques, have started to gain more and more relevance in predictive Physics (Karpatne et al., 2017a; Raissi et al., 2019; Li et al., 2019). The term coined for this new paradigm is Physics-Informed Machine Learning (PIML) (Karniadakis et al., 2021) or Scientific Machine Learning (SML) (Cuomo et al., 2022; Cueto and Chinesta, 2023).

As it is well-known, in any physical system, two types of state variables may be identified: (i) observable (measurable) ones that can be obtained directly from physical sensors such as position, temperature or forces; (ii) internal non-observable (not directly measurable) variables, that integrate locally other observable magnitudes and depend on the particular internal structure of the system, that is condensed and therefore lost in this integration process. This homogenization procedure is required in any "average" theory, established at scales higher than the one of quantum mechanics and first principles. In general, these internal state variables (e.g. stresses, plastic strains, damage, etc.) depend upon the whole time-history of the system, and collect the internal changes in the microstructure. Since they are not directly measurable, the only way of relating them with the former observable ones is by means of physical experiments, sufficiently simplified to allow assuming a certain internal state of the system (e.g. uniform distribution of stresses in the central section of a sample under uniaxial tension). The results obtained are extrapolated to more general conditions by additional assumptions (internal state models). In the DDS approach, the experimental results are used directly without appealing to that latter extrapolation procedure, thus avoiding using any state model with its inherent assumptions and associated errors. Of course, the counterpart is the need of big amounts of experimental data under sufficiently varied conditions. This is costly and does not avoid the need for simplified experimental designs and the implicit assumptions required to approximate the values of the internal state variables, since they are not directly measurable.

An opposite perspective is integrating physical knowledge into data science models to inform and improve the data prediction capability of neural networks, that is, to constrain the prediction domain of the standard data model to fulfill some physical requirements. This idea has been applied to different examples (Karpatne et al., 2017b; Raissi et al., 2017, 2019; Lu et al., 2021; Long et al., 2019; Bar and Sochen, 2019; Haghighat et al., 2021), while a classification of research topics was enumerated in Karpatne et al. (2017a). However, in all these works, the physical information was introduced directly as relations between the input and output layers. Only in some works a first attempt was made to provide the network with some explanatory capacity by adding as output some of the parameters associated with the internal state model (Raissi et al., 2019; Lu et al., 2021). A particular idea in this framework is using the physical universal laws to inform the neural network in such a way that it is then possible to associate some neuron values to internal state variables (observable or not). This idea, named as Physically-Guided Neural Networks with Internal Variables (PGNNIV), was recently introduced (Ayensa-Jiménez et al., 2021, 2022, 2024). In such method, the equations of evolution (physical principles) are treated as constraints between neuron values in the ANN, while the network adopts a particular architecture imposed by the Physics. Moreover, the internal state equations, that represent the averaged behavior of the internal structure of the system, are directly derived from the ANN outcome. This latter characteristic endows the methodology with explanatory capacity. The beauty and power of this idea is that only observable data are used while the internal variables may be predicted by the ANN itself, thus relaxing as much as we want the internal state model.

In particular, in continuum Physics (deformable solids and fluid mechanics, electromagnetism, energy and mass transport problems, etc.), the universal physical principles (ANN constraints) are written in terms of partial differential equations that are currently computationally solved by means of numerical methods like finite elements, boundary elements, finite volumes, meshless methods and a long etcetera (Ruas, 2016; Larsson and Thomee, 2009). These use a previous discretization step in space, driving to an algebraic, in general non-linear, system, that is then solved by means of standard matrix manipulation. Also, for time-dependent (evolution) problems, another discretization step in time is required to transform the time-continuum problem into a discrete one (alternatively automatic differentiation may be used (Raissi et al., 2019)). This includes the selection of a suitable time integrator (i.e. Euler, Multi-step, Runge–Kutta, among many others) (Larsson and Thomee, 2009). In the PGNNIV approach, solving this problem is straightforward by working with the discretized version of the problem, assigning an internal neuron to each nodal variable.

There are several reasons that supports the interest of using PGNNIV in continuum Physics: on one hand, the multiple problems of interest, in many disciplines, that are expressed in this framework. Secondly, the increasing use of images and videos as the main method for data provision in engineering and healthcare, for example, which is equivalent to having a continuous distribution of data in space, previously discretized (pixels in 2D images and voxels in 3D ones), and also in time (time frames in a video). In third place, and following this trend, in the last years, there has been a tremendous effort in treating images and in developing AI tools to make predictions from such images (e.g. convolutional neural networks - CNN-) (Yoo, 2015; Rawat and Wang, 2017; McCann et al., 2017; Anwar et al., 2018). Finally, the mathematical operators acting on the image can be extended to consider the standard differential operators in continuum Physics, thus allowing PGNNIV to leverage all current possibilities in image treatment to predict the evolution of a physical system (predictive capacity of PGNNIV) as well as to extract information on its structure (explanatory capacity of PGNNIV). Therefore, the similarity in the mathematical language of image treatment and image-based ANN predictions (convolutional filters) and of discretized continuous physical problems (discretized differential operators) is so high that makes it plausible to think of a common framework for both, which would allow to take profit of the tools available in both sides to improve the other.

The objective of this work is, therefore, to extend the PGNNIV methodology to continuum problems, showing its predictive capacity to get the input–output relation in a physical system from a sufficient set of data, as well as its unraveling (explanatory) ability to extract knowledge on the system internal structure, e.g. its microstructure. This is always performed considering the constraints imposed by Physics, and using only observable (measurable) variables in the training set. Also, the similarities between PDEs and CNN are highlighted and described in detail.

The structure of the paper is as follows: first, in Section 2, we introduce the problem to solve and the main objectives of the paper. Then, we present in Section 3 the general methodology to study continuum physics under the PGNNIV framework, that is, we reformulate the mathematical foundations of continuum physics using ANN formalism: both the fields and the operators are recast in a standard ANN language, that can be implemented in any Deep Learning (DL) platform. Next, we present in Section 4 several validation examples where the methodology is fully illustrated and its performance is analyzed when dealing with heterogeneous, anisotropic and nonlinear problems. The predictive and explanatory capacity of the methodology is here revealed. Finally, we finish the paper with a discussion and the main conclusions of the work. In order to ensure the reproducibility of the work, something fundamental in ML and in particular in Scientific ML, we provide an open source GitHub repository with all the presented computational studies: https://github.com/rmunozTMELab/PGNNIV-to-Continuum-Problems.git.

2. Theoretical background

The physical problem. Let us consider a certain physical problem defined by a set of (possibly nonlinear) partial differential equations which can usually be split into two main groups, universal physical laws and constitutive equations:

$$\mathcal{F}[\boldsymbol{u},\boldsymbol{v}] = \boldsymbol{f}, \tag{1a}$$

$$\mathcal{H}[\boldsymbol{u},\boldsymbol{v}] = \boldsymbol{0}. \tag{1b}$$

In the previous equation, $\mathcal{F}$ and $\mathcal{H}$ are differential operators that relate two unknown tensor field quantities, $\boldsymbol{u}$ and $\boldsymbol{v}$, with some other control quantities $\boldsymbol{f}$, also with their own tensor character. Eq. (1) must be completed with appropriate initial and/or boundary conditions to make the problem well-posed:

$$\mathcal{G}[\boldsymbol{u},\boldsymbol{v},\boldsymbol{g}] = 0, \tag{2}$$

where $\boldsymbol{g}$ is another known tensor field.

The functionals $\mathcal{F}$ and $\mathcal{G}$ represent the whole set of universal laws associated with the problem in hands, as well as particular geometrical and environmental constraints, whereas $\mathcal{H}$ represents all (possibly unknown) internal state or constitutive relations of the problem. Splitting the problem in these two sets of PDE systems also drives to the distinction between the two kind of unknown fields: the essential measurable fields, $\boldsymbol{u}$, and the internal state fields, $\boldsymbol{v}$, that are particular to each continuum based field theory. In many contexts, the underlying theory is formulated such that Eq. (1b) may be expressed in the form $\boldsymbol{v} = \mathcal{H}(\boldsymbol{u})$. For instance, in solid mechanics, $\mathcal{F}$ encodes mass, momentum and energy conservation equations, while $\mathcal{H}$ expresses the material-dependent constitutive relations between stresses and strains (or displacements) (Ayensa-Jiménez et al., 2024). Similarly, for heat transfer problems, $\mathcal{F}$ is related to energy conservation, whereas $\boldsymbol{v} = \mathcal{H}(\boldsymbol{u})$ is the Fourier's law for thermal conduction.

To fix ideas, and in the case of heat transfer theory, the governing equations of the problem are:

$$\rho c_p \frac{\partial u}{\partial t} = -\boldsymbol{\nabla}\cdot\boldsymbol{q} + \rho\boldsymbol{b}, \tag{3a}$$

$$\boldsymbol{q} = -\boldsymbol{K}\boldsymbol{\nabla}u, \tag{3b}$$

where $\boldsymbol{q}$ is the heat flux, $\boldsymbol{b}$ the heat source per unit mass, ρ the density field and c_p the specific heat capacity. The associated boundary conditions are:

$$u = \bar{u}, \quad \text{in} \quad \Gamma_D, \tag{4a}$$

$$\boldsymbol{q}\cdot\boldsymbol{n} = \bar{q}, \quad \text{in} \quad \Gamma_N. \tag{4b}$$

where $\partial\Omega = \Gamma_D \cap \Gamma_N$ is the domain boundary, $\bar{u}$ is the prescribed temperature field at the boundary region Γ_D and $\bar{q}$ is the prescribed heat flow at the boundary part Γ_N.

Eq. (3a) represents the energy conservation, which is a fundamental law. On the contrary, Eq. (3b) is the constitutive relation, which is postulated in this model as a linear relationship between the temperature gradient and the heat flow, a phenomenological relation that propagates the microscopic information of the considered material to the macroscopic level. This example may be enriched when considering nonlinearities, keeping the structure of Eq. (1). Similarly, one could consider infinitesimal or finite solid mechanics, and distinguish between the universal laws (the conservation of linear and angular momenta as well as the kinematic relations), and the constitutive relation between stresses and strains (Bonet et al., 2016), as suggested by some recent works (Ayensa-Jiménez et al., 2024). The general mixed boundary constraints given by (4) may be considered as special cases of more general functional operators. For instance $\mathcal{G}[\boldsymbol{U}]$ may be defined as

$$\mathcal{G}[u,\boldsymbol{q}](\boldsymbol{x}) = \begin{cases} \int_\Omega (u(\boldsymbol{y}) - \bar{u}(\boldsymbol{y}))\,\delta(\boldsymbol{x}-\boldsymbol{y})\,\mathrm{d}\boldsymbol{y}, \ \forall \boldsymbol{x}\in\Gamma_D, \\ \int_\Omega (\boldsymbol{q}(\boldsymbol{y})\cdot\boldsymbol{n} - \bar{q}(\boldsymbol{y}))\,\delta(\boldsymbol{x}-\boldsymbol{y})\,\mathrm{d}\boldsymbol{y}, \ \forall \boldsymbol{x}\in\Gamma_N, \end{cases}$$

where δ is the Dirac delta function. Hence, for this particular problem, u is the essential measurable field and $\boldsymbol{q}$ is an internal field, which corresponds to $\boldsymbol{u}$ and $\boldsymbol{v}$ respectively in Eq. (1), arising from classical field theory. Finally, $\rho\boldsymbol{b}$ corresponds to the stimulus $\boldsymbol{f}$, and $\bar{u}$ and $\bar{q}$ are the known values of the respective Dirichlet and Neumann prescribed boundary conditions identified with $\boldsymbol{g}$ in Eq. (2).

To solve numerically the physical problem (1) with boundary conditions (2), it has to be previously discretized both in space and time by means of one of the many discretization techniques available (Larsson and Thomee, 2009), such as the Finite Difference Method (Langtangen, 1999) (FDM), Finite Element Method (Zienkiewicz and Morice, 1971) (FEM), or other spectral techniques (Boyd, 2001). Once discretized, the physical problems writes:

$$\boldsymbol{F}(\boldsymbol{u}_{(n)},\boldsymbol{v}_{(n)}) = \boldsymbol{f}_{(n)}, \tag{5a}$$

$$\boldsymbol{H}(\boldsymbol{u}_{(n)},\boldsymbol{v}_{(n)}) = \boldsymbol{0}, \tag{5b}$$

with boundary conditions:

$$\boldsymbol{G}(\boldsymbol{u}_{(n)},\boldsymbol{v}_{(n)},\boldsymbol{g}_{(m)}) = 0, \tag{6}$$

where now $\boldsymbol{u}_{(n)}$, $\boldsymbol{v}_{(n)}$ are the nodal/spectral representation of the unknown respective tensor fields, being n the number of degrees of freedom of the problem, $\boldsymbol{f}_{(n)}$ and $\boldsymbol{g}_{(m)}$ are the nodal/spectral representations of the known tensor fields, with m the number of prescribed degrees of freedom at the boundary, and $\boldsymbol{F}$, $\boldsymbol{G}$ and $\boldsymbol{H}$ are corresponding array-valued functions. As commented before, often, we replace Eq. (5b) by $\boldsymbol{v}_{(n)} = \boldsymbol{H}(\boldsymbol{u}_{(n)})$. Note that in this discretized version of the problem, the continuous position label $\boldsymbol{x}$ is replaced by a discrete index j (e.g. $\boldsymbol{u}_j = \boldsymbol{u}(\boldsymbol{x}_j)$).

The machine learning problem. Alternatively to the physical problem, we can think of the formulation above as a ML problem consisting of a collection of input–output data $\mathcal{D} = \{(\boldsymbol{x}^i,\boldsymbol{y}^i),\ i = 1,\dots,N\}$ and whose goal is to learn the underlying implicit relationship $\boldsymbol{y} = \boldsymbol{Y}(\boldsymbol{x})$, or, in other words, to build an estimate $\hat{\boldsymbol{y}}$ from $\boldsymbol{x}$. To face this problem it is possible to set up a DL regression model relating $\boldsymbol{x}$ and $\boldsymbol{y}$ that may be expressed as $\hat{\boldsymbol{y}} = \mathsf{Y}(\boldsymbol{x})$ (note that the use of sans serif notation indicates that there is a DL model relating these two variables). Once the error $\boldsymbol{e} = \hat{\boldsymbol{y}} - \boldsymbol{y}$ is defined, the construction of the model Y is performed

by solving a minimization problem. We usually define a loss function related to the norm of the error for the whole learning dataset $\mathcal{D}$, for instance $\mathcal{L}_{\text{data}} = \sum_{i=1}^{N} \|\boldsymbol{e}^i\|^2$, with $\boldsymbol{e}^i = \mathsf{Y}(\boldsymbol{x}^i) - \boldsymbol{y}^i$.

3. Methods

3.1. Coupling physics and data science problems

To link both the Physics-based and ML problems we build a Physically-Guided Neural Network with Internal Variables (PGNNIV), following the three steps described below. Additional details on the PGNNIV methodology may be found in Ayensa-Jiménez et al. (2021, 2024):

1. **Selection of appropriate input and output variables:** Since we consider that they come from any type of data-capturing sensor or device, we require that these variables have to be measurable, so they are a subset of variables $\boldsymbol{u}_{(n)}$, $\boldsymbol{f}_{(n)}$ and $\boldsymbol{g}_{(m)}$ of the problem. Of course, the output variables are always the variables we want to predict. For notation purposes, we write $\boldsymbol{x} = \boldsymbol{I}(\boldsymbol{u}_{(n)}, \boldsymbol{f}_{(n)}, \boldsymbol{g}_{(m)})$ and $\boldsymbol{y} = \boldsymbol{O}(\boldsymbol{u}_{(n)}, \boldsymbol{f}_{(n)}, \boldsymbol{g}_{(m)})$.
2. **Physical constraints:** The physics of the problem given by Eq. (5a) is now supplied to the ANN as constraints on some prescribed layers (PILs). However, this equation includes the internal variables $\boldsymbol{v}_{(n)}$, so for the problem to make sense, we have to include Eq. (5b) as an element of the network. For example, by defining $\boldsymbol{v}_{(n)} = \boldsymbol{H}(\boldsymbol{u}_{(n)})$.
3. **Model relaxation:** Since the interest of this methodology is both *to predict* new values of the variable $\boldsymbol{y}$ and *to unravel* the constitutive model $\boldsymbol{H}$, this latter is generally only partially known. That is, we may know partly its functional structure, or some of the associated parameters. Therefore, the model is replaced by a subnetwork, for instance, $\boldsymbol{v}_{(n)} = \mathsf{H}(\boldsymbol{u}_{(n)})$. Some guidelines to the set-up of this H are given hereafter in Section 3.3.

These three steps have to be balanced and consistent: each model has to be complemented with the definition of supplementary output variables and/or the addition of physical constraints to learn both the output variables and the constitutive model. Besides, these constraints can be associated to universal balance laws or to extra knowledge on the system, such as frame invariance, material symmetries and so on.

Now, all ingredients of the PGNNIV approach have been already set-up: we have a predictive input–output neural network, that we call the predictive network, with appropriate physical constraints acting on some Prescribed Internal Layers (PILs). Renaming all the constraints associated with the known physics of the problem as $\boldsymbol{R} := (\boldsymbol{R}_1, \boldsymbol{R}_2, \dots, \boldsymbol{R}_k)$, that is, the whole set of relations in $\boldsymbol{F}$ and $\boldsymbol{G}$ and, if desired, part of the relations in $\boldsymbol{H}$, or any other knowledge on the system, it is possible to write the PGNNIV problem as:

$$\begin{aligned} &\boldsymbol{y} = \mathsf{Y}(\boldsymbol{x}); \quad \boldsymbol{v}_{(n)} = \mathsf{H}(\boldsymbol{u}_{(n)}), \\ &\text{s. t.} \quad \boldsymbol{x} = \boldsymbol{I}(\boldsymbol{u}_{(n)}, \boldsymbol{f}_{(n)}, \boldsymbol{g}_{(m)}), \\ &\qquad \boldsymbol{y} = \boldsymbol{O}(\boldsymbol{u}_{(n)}, \boldsymbol{f}_{(n)}, \boldsymbol{g}_{(m)}), \\ &\qquad \boldsymbol{R}(\boldsymbol{u}_{(n)}, \boldsymbol{v}_{(n)}, \boldsymbol{f}_{(n)}, \boldsymbol{g}_{(m)}) = \boldsymbol{0}. \end{aligned} \tag{7}$$

Following the standard approach in ANN, a cost function is minimized, including now the physical constraints as penalty terms:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \alpha \mathcal{L}_{\text{physics}} + \beta \mathcal{L}_{\text{model}}, \tag{8}$$

where with $\mathcal{L}_{\text{physics}}$ we refer to the physics associated with universal laws and with $\mathcal{L}_{\text{model}}$ to the ones associated with additional model constraints and assumptions.

When the loss functions is expressed in the form of Eq. (8), we say that the constraints are formulated in weak form. It is of course possible to define the network architecture of the components H or even Y in such a way that the constraints are identically satisfied. In that case we say that the constraints are formulated in strong form. Many ways have been proposed to define the ANN architecture in such a way that physical or geometrical constraints are satisfied identically (Greydanus et al., 2019; Jin et al., 2020; Ayensa-Jiménez et al., 2022; David and Méhats, 2023). Here we follow the more general approach of including them in their weak form to adapt the methodology to more general contexts. Of course, it is possible to combine different constraints in strong and weak form.

Defining a loss function such as the one given by Eq. (8) is equivalent to consider a physically augmented neural network where a new set of output variables are identically equal to zero and are related to the PILs through predefined relations. For instance, using the L_2 norm, and defining $e = \|\boldsymbol{e}\|$ and $\pi_j = \|\boldsymbol{R}_j\|$ leads to the minimization of the cost function:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \left((e^i)^2 + \sum_{j=1}^{r} c_j (\pi_j^i)^2 \right), \tag{9}$$

where the upper index i indicates that the values of e and π_j are associated with the sample i of the dataset $\mathcal{D}$, and c_j are penalty coefficients. Using a standard ML notation, we can write

$$\mathcal{L} = \text{MSE}(e) + \sum_{j=1}^{r} c_j \text{MSE}(\pi_j). \tag{10}$$

Note that the coefficients c_j, $j = 1, \dots, r$, are numerical parameters of the optimization procedure, so in the ANN framework, they are new hyperparameters of the learning process. Eq. (10) may be written in a more compact form as

$$\mathcal{L} = \sum_{j=0}^{r} c_j \text{MSE}(\pi_j), \tag{11}$$

just by adding a penalty coefficient to the loss term c_0 and defining $\pi_0 = e$.

3.2. Data and fields description

In continuum physical problems, a time-dependent tensor field is a point-dependent tensor $\boldsymbol{f} = \boldsymbol{f}(\boldsymbol{x}, t)$ indexed therefore by the point coordinate $\boldsymbol{x}$ and the time t. Once discretized, these fields may be represented by arrays of appropriate dimension. For example, the time dependent (l-covariant, m-contravariant) tensor field $\boldsymbol{f}$ is represented by the multi-indexed array $F^{i_1,\dots,i_l}_{j_1,\dots,j_m}|^{k_1,k_2,k_3,k_4} = F^{i_1,\dots,i_l}_{j_1,\dots,j_m}(x_{k_1}, y_{k_2}, z_{k_3}, t_{k_4})$. Note that the second set of contravariant indexes are associated with the spatial coordinates and time (voxels and time frames when referring to a particular image or video).

The TensorFlow framework (Abadi et al., 2016) is particularly suitable for working with data associated with discretized fields. Indeed, a tensor field $\boldsymbol{f}$ is represented in TensorFlow notation by a multiarray[1] `f`. When that tensor field varies among samples of a given dataset, the array rank is expanded to take this into consideration. For instance, the value of a two dimensional discretized displacement field at a given time $t_k = k\Delta t$, $\boldsymbol{U}(\boldsymbol{x}, t_k)$, for a given sampled value i, is represented by `U[·,·,k]·|i|`, that is a 3rd-rank array, where the first two symbol "·" represents the two spatial indexes as a whole and the last one the two vector components as a whole.

One main feature of ML frameworks is that they allow working with both data-independent and data-dependent tensors. Data-independent tensors are unalterable over the learning dataset, whereas data-dependent tensors depend on the considered sample. For instance, for the heat transfer problem, the temperature field u and the energy flux $\boldsymbol{q}$ are dependent on the boundary conditions, $\boldsymbol{g}$, that may change with the input of each problem. On the contrary, the thermal conductivity tensor for a given material is constant for any possible input stimulus (internal forces and boundary conditions) even if we consider the material as heterogeneous, so that such tensor is spatially-dependent.

[1] In this work, we distinguish between an array (set of numbers spatially arranged for computational purposes) and a proper tensor, when it belongs to a tensor space and usually has its own meaning in a physical context.

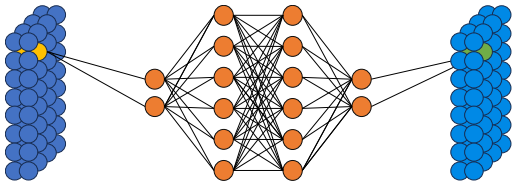


Fig. 1. Moving Multilayer Perceptron (mMLP) scheme.

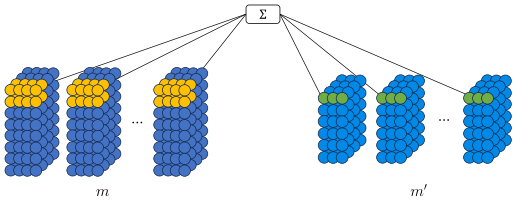


Fig. 2. Convolutional local linear operator (CNN) scheme.

3.3. Operator description

The operator $\mathcal{F}$ on a vector field $\boldsymbol{u}$ acts, after discretization, as a vector function $\boldsymbol{F}$. This is directly reframed in tensor language by defining a function relating two multiarray tensors. Briefly, a functional relationship $\boldsymbol{v} = \mathcal{F}(\boldsymbol{u})$ is first discretized into a function of n variables, $\boldsymbol{v}_{(n)} = \boldsymbol{F}(\boldsymbol{u}_{(n)})$, which in turns is expressed as a tensor relationship $\mathtt{v} = \mathtt{F}(\mathtt{u})$. Nonetheless, there are two fundamental observation that has to be mentioned:

1. First, most of the operators acting in the formulation of continuum physics are either (i) functions acting over the field values or (ii) linear functional operators. Even more, a vast majority of the linear operators involved in the formulation of continua are local operators. Our framework is adapted to that purpose, as both classes of operators may be seen as convolution filters.

   (a) The first case, that is an operator such that $v_i = f(u_i)$, may be expanded using convolution filters in the tensor framework into a moving MultiLayer Perceptron (mMLP). For example, if $\boldsymbol{u}$ and $\boldsymbol{v}$ are 2D vector fields, then, $u^{i_1}|^{k_1,k_2}$ and $v^{i_1}|^{k_1,k_2}$ are rank 3 arrays and an operator relating them is conceptually illustrated in Fig. 1. The universal approximation theorem (Cybenko, 1989; Hornik, 1991; Lu et al., 2017) guarantees that every regular enough function f may be approximated by multilayer perceptrons so this approximation makes sense.

   (b) The second case, that is local linear operators, may be reframed in the tensor framework using convolution filters of a given size. If $\boldsymbol{v} = \boldsymbol{F}(\boldsymbol{u})$ is a local linear operator relating a rank k tensor $\boldsymbol{u}$ and a rank k' tensor $\boldsymbol{v}$, both representing 2D spatial fields $\boldsymbol{F}$ can be encoded using a convolutional filter. Fig. 2 illustrates a possible relation for two dimensional tensor fields, represented using arrays of size $[n_x, n_y, 4, m]$. It is important to note that, as the considered operator is local, the spatial field $\boldsymbol{v}$ is undefined at some values close to the boundaries, so $n'_x < n_x$ and $n'_y < n_y$.

2. Among the data-independent tensors, we distinguish between constant (non-trainable) and variable (trainable) tensors. Once we have fixed the physical problem and decided what is the input–output relation that has to be learned, the role of each operator tensor becomes natural: when a tensor is involved in a known operator, such as the ones related with $\boldsymbol{F}$ and $\boldsymbol{G}$ functions, it is a constant array and will be denoted here with a star. One particular example is the tensor associated with the gradient operator. If the tensor is associated with an unknown relationship, such as the predictive network $\mathtt{Y}$ or the model network $\mathtt{H}$, the tensor is represented using a variable array. An example of a variable tensor is the (possibly heterogeneous) 4th order elastic tensor $\mathtt{C}$ represented by the 7th order array $\boldsymbol{C}^{ijkl}|^{pqr}$.

3.3.1. Brief taxonomy of operators

To illustrate the concepts introduced above, we particularize such general ideas to a brief taxonomy of different operators found in continuum physical problems. This discussion is not intended to be exhaustive and complete, but shows the suitability of the PGNNIV formulation to handle a wide range of operators.

Linear differential operators. With the presented framework, all differential operators can be cast as pre-defined filters acting on field tensors. A (discretized) differential operator is a function $\mathtt{D}$ transforming one multiarray into another. For linear differential operators, $\mathtt{D}$ inherits the linearity. Consequently, they may be encoded using known constant tensors $\mathtt{D}^*$. To fix ideas, let us consider the equilibrium equation in solid mechanics involving covariant derivative, as $\boldsymbol{\nabla}$ is a linear differential operator defined for a two contravariant tensor. If $\boldsymbol{f} = \boldsymbol{\nabla} \cdot \boldsymbol{X}$, $f^i = X^{ij}_{;j}$, where ";" represents the covariant derivative. In a coordinate representation, the covariant derivative is expressed for the considered tensor as $X^{ij}_{;j} = \partial_j X^{ij} + \Gamma^i_{jk} X^{kj} + \Gamma^j_{jk} X^{ik}$ where Γ^k_{ij} are the Christoffel symbols that, for the Levi-Civita connection, are defined in terms of the metric tensor $\mathfrak{g}$, and satisfy the following linear equation $\mathfrak{g}_{kl}\Gamma^k_{ij} = \frac{1}{2}\left(\mathfrak{g}_{jl,i} + \mathfrak{g}_{li,j} - \mathfrak{g}_{ij,l}\right)$. Now, as $\mathfrak{g}$ encodes the geometry of the problem, the only ingredient to reframe equilibrium equation to the multiarray framework is to select a discretization of the common, one dimensional, derivative operator ∂_i as an array operator.

Recall that all linear differential operators may be reframed as convolutional filters in the spatial slots. This has important consequences from a practical point of view:

- Convolution operators are sparse in the discretization dimensions (i.e., those of greater dimensionality). This allows sparse-based algebra and storage, resulting in high-performance computations and less demanding requirements.
- They may be easily built and used in standard ANN software codes and tools, such as TensorFlow, although some care must be taken in indexing.

In summary, all differential operators involved in the fundamental balance equations in Continuum Physics (universal laws) may be encapsulated in this framework, provided that we have established two main ingredients: the space geometry ($\mathfrak{g}$) and a given discretization rule for differentiation (∂).

Constitutive models. Constitutive models (or internal state equations) define the internal state variables (in general, non-measurable) of the problem in terms of the essential (measurable) ones (Ayensa-Jiménez et al., 2021). They can be written in a quite general case as $\boldsymbol{v} = \mathcal{H}(\boldsymbol{u})$, where $\boldsymbol{v}$ is the set of internal variable fields and $\boldsymbol{u}$ is the set of essential variable fields (for instance, stresses and displacements in continuum mechanics, macroscopic -$\boldsymbol{D}, \boldsymbol{H}$- and microscopic -$\boldsymbol{E}, \boldsymbol{B}$- intensity of electromagnetic fields in electromagnetism or temperatures and energy fluxes in heat transfer problems, among many other examples) and $\mathcal{H}$ must be interpreted as a functional e.g. $\boldsymbol{q} = \mathcal{H}(u)$ for the heat transfer problem). Once discretized, the constitutive equation is expressed as $\boldsymbol{v}_{(n)} = \boldsymbol{H}(\boldsymbol{u}_{(n)})$, where now $\boldsymbol{v}_{(n)}$ and $\boldsymbol{u}_{(n)}$ are the tensor fields associated with the nodal field values and $\boldsymbol{H}$ is a (in general nonlinear) mapping.

All symmetries and simplifications that can be assumed in the constitutive equation relating the two internal variable fields may be transcribed to the structure of the functional $\mathcal{H}$ and inherited by the function $\boldsymbol{H}$:

- **Linearity:** The linearity of the functional $\mathcal{H}$ is translated directly into the linearity of the function $\boldsymbol{H}$. Using ANN language, this is equivalent to build a network with no internal layers between the neurons corresponding to u and v, associated with the fields considered, as well as no activation functions. Different levels of complexity and non-linearity of $\mathcal{H}$ (and therefore $\boldsymbol{H}$) may be handled with appropriate architectures of the deep neural network relating $\boldsymbol{v}$ and $\boldsymbol{u}$.
- **Sparsity:** The structure of the function $\boldsymbol{H}$ is further exploited using the ANN architecture, involving different levels of sparsity (see Fig. 3):
  - **Local operators:** Local constitutive laws, such that the operator $\mathcal{H}$ is local, that is, the value of $\boldsymbol{v}$ depends on the values of $\boldsymbol{u}$ in a neighborhood of $\boldsymbol{x}$. For differential operators, this means that

    $$\mathcal{H}(\boldsymbol{u}) = \boldsymbol{H}(D^0[\boldsymbol{u}](\boldsymbol{x}), D^1[\boldsymbol{u}](\boldsymbol{x}), \dots, D^m[\boldsymbol{u}](\boldsymbol{x})),$$

    where D^k is a differential operator of order k, being D^0 the identity. $m < \infty$ is called the order of the locality.
    When $m = 0$ we speak about order-zero local or point-wise constitutive laws. In that case $\boldsymbol{v}(\boldsymbol{x}) = \boldsymbol{H}(\boldsymbol{u}(\boldsymbol{x}))$, or, using the array notation, $\frac{\partial v^{\bullet}_{\bullet}|^I}{\partial u^{\bullet}_{\bullet}|^J} = \delta^I_J$, being I and J spatial multi-indexes. This entails *block-diagonal* array structures, in the slots associated with spatial–temporal discretization. For instance, for 3D evolution problems, v[$i, j, k, l|\cdot$]=H(u[$i, j, k, l|\cdot$]) where the indices i, j, k and l are used to encode the value of the field $\boldsymbol{v}(x_i, y_j, z_k, t_l)$. In the DL framework, these types of relationships are associated with moving operators, as illustrated in Fig. 3(a). When $m > 0$, the operators are sparse but not necessarily *block-diagonal*. In the DL framework, these operators are associated with CNN, as shown in Fig. 3(b).
  - **Non-local operators:** In non-local constitutive laws, the value of $\boldsymbol{v}$ cannot be determined solely from the values of $\boldsymbol{u}(\boldsymbol{x})$ in any neighborhood of $\boldsymbol{x}$. For instance, $\boldsymbol{v}(\boldsymbol{x})$ could depend on the values of $\boldsymbol{u}$ on the whole spatial domain. For differential operators,

    $$\mathcal{H}(\boldsymbol{u}) = \boldsymbol{H}(D^0[\boldsymbol{u}](\boldsymbol{x}), \dots, D^m[\boldsymbol{u}](\boldsymbol{x}), \dots).$$

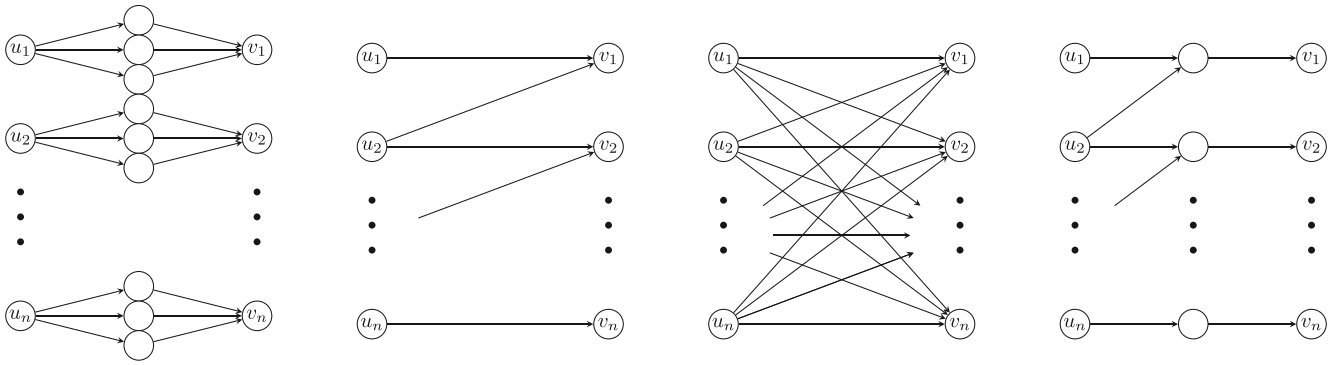
    There are many ways of defining non-local functionals (see for instance (Ros-Oton, 2015) and included references for a motivation and examples with elliptic operators). In that case, the tensors are dense and so it is the architecture of the ANN associated with the model, as illustrated in Fig. 3(c).

  Obviously, these different situations may be modulated in several hierarchical levels in the network. For instance, in linear elasticity, the material is local (order 1) with respect to the displacement field. Therefore, an accurate ANN architecture for working with this model is obtained by combining the previous ideas, as shown in Fig. 3(d).

- **Spatial or temporal invariance:** The difference between homogeneous and heterogeneous constitutive relations may be also exploited. For local models (block-diagonal or sparse operators), the different blocks or filters may be the same or may be dependent on the spatial considered point, with a prescribed symmetry. For the former, the indexes referring to the spatial part of the tensor are spurious and therefore may be omitted. For instance, in the heat transfer problem, in general, $q^i|^I = K^{ij}|^I_J u_{,j}|^J$. The fact that we implicitly assume that $\boldsymbol{q}(\boldsymbol{x}) = \boldsymbol{H}(\nabla u(\boldsymbol{x}))$ (point-wise with respect to the temperature gradient) implies that $K^{ij}|^I_J = C^{ij}$ if $I = J$ and 0 otherwise so we can write $K^{ij}|^I$. For homogeneous materials, $K^{ij}|^I = K^{ij}$.
- **Further symmetries:** The array relations may be adapted for the exploitation of further symmetries or structure of the constitutive equation. This includes:
  - Relations derived from the principle of objectivity, that is, reference frame independence. Many works detail how to enforce these features in the ANN architecture (Ling et al., 2016; Heider et al., 2020; Xu et al., 2021; Garanger et al., 2024).
  - Constraints related to the physical or geometrical foundations of the model (e.g. major and minor symmetries of the elastic tensor, associated with thermodynamic constraints, angular momentum conservation, compatibility constraints). These ideas have already been used in macroscopic contexts (Masi et al., 2021) and multiscale problems (Masi and Stefanou, 2022).
  - Additional constraints related to special symmetries of the constitutive model, that is, orthotropy, isotropy (Fernández et al., 2021).

  All these symmetries may be enforced by adding constraints to the PGNNIV (that is, in an implicit way or weak form) or by assuming a given architecture for the neural network operator (explicit or strong form). Indeed, if $\boldsymbol{v} = \boldsymbol{H}(\boldsymbol{u})$, the existence of a given symmetry is equivalent, in the Noether sense, to the action of a given group of transformations, so that $\boldsymbol{H}(\boldsymbol{u}) = \boldsymbol{H}(\boldsymbol{A}(\boldsymbol{u}))$ where $\boldsymbol{A} \in \mathcal{A}$, a group of transformations. Therefore, we can look for a finite set of transformations $\boldsymbol{A}_k$ in a way such that $\boldsymbol{H}(\boldsymbol{u}) = \boldsymbol{H}(\boldsymbol{A}(\boldsymbol{u})), A \in \mathcal{A} \Leftrightarrow \boldsymbol{A}_k(\boldsymbol{H}(\boldsymbol{u})) = \boldsymbol{H}(\boldsymbol{u})$ or to *a priory* set up an architecture for the ANN so that it is invariant under the action of all $\boldsymbol{A}$ in what is nowadays known as Geometric Deep Learning (Bronstein et al., 2017).
- Finally, parametric fitting is a very particular case, in which some of the internal layers are related to the others by means of a parametric explicit expression or equivalently by prescribing the related variables to lie on a given prescribed manifold. In that case, $\boldsymbol{v} = \boldsymbol{H}(\boldsymbol{u}; \lambda)$ where the function $\boldsymbol{H}$ is explicitly imposed, and the functional relationship depends on the value of unknown parameters λ, that are variable (trainable) scalars adjusted, in general, during the training process. This approach recovers the classical approach when a given constitutive model structure is assumed, and is either useful for prediction problems and for model selection or validation (Ayensa-Jiménez et al., 2021)

Numerical filters. Sometimes it is useful to introduce other operators to enforce higher-order discretizations, or to adapt the problem to other numerical methods. This is the case of special filters for meshless approaches such as SPH (Gingold and Monaghan, 1977), DEM (Nayroles et al., 1992) or NEM (Sukumar et al., 1998) among a crowd. Also, it is easy to adapt this framework to integral formulations as in the FEM. Indeed, finite element integrals may be expressed in terms of the nodal values, being this relationship dependent on the shape function and the chosen numerical integrator, but otherwise fixed for a given degree of approximation. For instance, a moving averaging filter applied to the nodes recovers the framework of linear shape functions for a given element. It is also possible to increase the order of the differential operators. This relies on the fact that a higher-order differential operator can be expressed as the subsequent application of lower-order ones, enriching the differentiation scheme. For instance, if Δ^+_h is the forward difference operator ($\Delta^+_h f^i = \frac{1}{h}(f^{i+1} - f^i)$), of order h, $\frac{1}{h}(\Delta^+_h - \frac{1}{2}(\Delta^+_h)^2)$ is a forward difference operator of order h^2. Finally, another useful possibility is that of stabilization filters in time-dependent problems to ensure the fulfillment of well-known stability criteria (Fischer et al., 2001) or the filters that are designed for obtaining and high fidelity time integrations, such as Crank–Nicolson integration or Runge–Kutta integrators (Raissi et al., 2019).



(a) Pointwise constitutive relation. (b) Local constitutive relation. (c) Non-local constitutive relation. (d) Hierarchical combination.

Fig. 3. Illustration of the different structures associated with constitutive equations. The different schemes are illustrative for one-dimensional problems. Note that when convolutional layers are applied (local operators), values at the boundaries may not be conveniently described.

Probes and sensors. The last application of filters are probes. Probes are measurable or quantifiable values related to the different fields by a known function. The most common probe is the value of a tensor field evaluated at a point or a region. Other common probes are measurements defined in the Data-Driven context (Ayensa-Jiménez et al., 2019), such as surface stress forces over a plane or strains along a direction, at a certain point. These values may be expressed in terms of data-dependent or data-independent tensor quantities (contractions with other vector/tensors, trace, etc.). As a particular case, we may consider integral quantities such as the flux of a tensor field over a surface (mass, momentum or energy flux), related to conserved quantities in field theories (Noether charges).

4. Case study: stationary diffusion equation

4.1. Problem formulation

The next example illustrates the use of PGNNIV in continuum physical problems after discretization. Let us suppose the following partial differential equation corresponding to a diffusion problem:

$$-\boldsymbol{\nabla}\cdot(\boldsymbol{K}\boldsymbol{\nabla}u)=f, \tag{12}$$

where u is the solution field and f is the source term. This problem is ubiquitous in physics and engineering. Indeed, Eq. (12) is used for instance in stationary heat transfer conduction problems with u the temperature field, in steady-state water seepage in solid mechanics, or in electrostatics, among many others. Eq. (12) is the combination of two different laws:

- **A fundamental functional principle** as it is energy conservation (heat transfer), mass conservation (species diffusion) or Gauss law (electrostatics), that states as $\boldsymbol{\nabla}\cdot\boldsymbol{q}=f$ where $\boldsymbol{q}$ is the flux vector (heat flux, mass flux or electric displacement field) and f is the source term (heat source, mass source or electric charge density).
- **A constitutive functional equation** as it is the Fourier law (heat transfer), Fick's law (diffusion) or dielectric behavior (electrostatics), relating the flux variable $\boldsymbol{q}$ that plays the role of internal state field (non-measurable if no additional assumption is made, e.g. uniform distribution of the transported magnitude through a certain area), with the essential field u. Commonly, this relationship is formulated in tensor form as $\boldsymbol{q}=-\boldsymbol{K}\boldsymbol{\nabla}u$, where $\boldsymbol{K}$ is the (thermal) conductivity tensor, the diffusion tensor, or the dielectric permittivity tensor, respectively, depending on the particular physical problem considered. For heterogeneous problems, $\boldsymbol{K}$ may depend on the spatial coordinate $\boldsymbol{x}$. Besides, for general nonlinear problems, the tensor $\boldsymbol{K}$ may be dependent (in a functional sense) on the field u.

With these assumptions, Eq. (12) may be split in:

$$\boldsymbol{\nabla}\cdot\boldsymbol{q}=f, \tag{13a}$$

$$\boldsymbol{q}=-\boldsymbol{K}\boldsymbol{\nabla}u, \tag{13b}$$

together with appropriate boundary conditions.

Using the framework described above, Eq. (13a) is the universal law of the problem, and Eq. (13b) is the internal state equation. Here, the fundamental principle and the state equation are expressed in functional form. But this subtlety is bypassed by using any common discretization technique such as FEM or FDM so the values of u, $\boldsymbol{q}$ and $\boldsymbol{K}$ are replaced by the corresponding interpolating (nodal) values or by the approximation values, $u_{(n)}$, $\boldsymbol{q}_{(n)}$ and $\boldsymbol{K}_{(n)}$, depending on the particular approach.

For instance, for two-dimensional problems, using forward first-order finite differences, a discretized version of Eqs. (13) using the notation presented in Section 3 is:

$$\frac{q_1|^{i+1,j}-q_1|^{i,j}}{L_1}+\frac{q_2|^{i,j+1}-q_2|^{i,j}}{L_2}=f|^{i+1,j+1}, \tag{14a}$$

$$q_1|^{i,j}=-k_{11}|^{i,j}\frac{u|^{i+1,j}-u|^{i,j}}{L_1}-k_{12}|^{i,j}\frac{u|^{i,j+1}-u|^{i,j}}{L_2}, \tag{14b}$$

$$q_2|^{i,j}=-k_{21}|^{i,j}\frac{u|^{i+1,j}-u|^{i,j}}{L_1}-k_{22}|^{i,j}\frac{u|^{i,j+1}-u|^{i,j}}{L_2}, \tag{14c}$$

with $i=1,\dots,n_x \quad j=1,\dots,n_y$, where L_k $(k=1,2)$ is an appropriate mesh size, $u|^{i,j}$ are the field variables, $q_k|^{i,j}$ $(k=1,2)$ are the internal state variables and the functional relationship $q=\mathcal{H}(u)$ is now in the form of an algebraic equation $\boldsymbol{q}_{(n)}=\boldsymbol{H}\left(\boldsymbol{u}_{(n)}\right)$, where we have defined

$$\boldsymbol{q}_{(n)}=(q_i|^{rs})=(q_1|^{11},\dots,q_1|^{n_xn_y},q_2|^{11},\dots,q_2|^{n_xn_y}),$$
$$\boldsymbol{u}_{(n)}=(u|^{rs})=(u|^{11},\dots,u|^{n_xn_y}),$$
$$f_{(n)}=(f|^{rs})=(f|^{11},\dots,f|^{n_xn_y}).$$

Of course, when solving Eqs. (13), or their corresponding discrete versions Eqs. (14), proper boundary conditions, parametrized in terms of a set of variables $\boldsymbol{g}_{(m)}$, must be supplied. If the problem is now formulated within the PGNNIV framework, and for many cases, these latter boundary values, together with $\boldsymbol{f}_{(n)}$, are the natural inputs of the problem, being $\boldsymbol{u}_{(n)}$ the output ones.

It is possible to predict the value of the fields u, $\boldsymbol{q}$ and $\boldsymbol{K}$ given the fields f and g by using the approach stated in Section 3 such that the nodal values of u (that is $\boldsymbol{u}_{(n)}$) are learned from a sufficiently and varied dataset of input–output pairs.

For illustrative purposes let us consider the problem given by Eqs. (14) with boundary conditions

$$u|^{0,y}=g_1,\quad u|^{n_x,y}=g_2,\quad u|^{x,0}=g_3,\quad u|^{x,n_y}=g_4. \tag{15}$$

Eqs. (14) are formulated so that the internal state variables verify the fundamental principle of flux conservation given by Eqs. ((14b),

(14c)) and the specific boundary conditions given by Eqs. (15). At this level, the main problem relies on the form of the functions $\boldsymbol{K}_{(r)} = \boldsymbol{K}_{(r)}(\boldsymbol{u}_{(1)}, \dots, \boldsymbol{u}_{(n)})$, with $r = 1, \dots, n$, that is a multiple input–multiple output relationship that will be learned using DL regression techniques. Further assumptions can be made about the functional form of this relationship, that may be translated to the structure of the subnetwork associated to the constitutive equation as detailed in Section 3. For example:

- Assuming a local relationship of order m between q and u, that is,

  $$q_{ij}|^{rs} = F_{ij}\left(D^m\left(u|^{rs}\right), D^{m-1}\left(u|^{rs}\right), \dots, D^0\left(u|^{rs}\right)\right),$$

  where D is a discretized difference operator (that is, a gradient filter) and $D^0\left(u|^{rs}\right) = u|^{rs}$. Furthermore, it is possible to extend the methodology for non-local operators (Ciaurri et al., 2018), with the inconvenience of the numerical and computational complexity. In particular, a nonlinear relationship may be reduced to a separable form involving the field u, the gradient of the field u, and higher-order derivatives, or even non-local operators. Thus, $q_i|^{rs} = \Pi_{j=1}^{k} F_{ij}\left((D)^{r_j}\left(u|^{rs}\right)^{s_j}\right)$ where now F_{ij} are the functions to be learned. Although this functional form seems arbitrary to some extent, it is ubiquitous in mathematical (Osserman, 2013), physical (Frank, 2005), engineering (Barenblatt et al., 1989) and financial (Barles and Soner, 1998) problems as it involves a broad class of problems.
- Assuming a linear (possibly heterogeneous) relationship between the gradient of u and the flux $\boldsymbol{q}$. In that case, $q_i|^{rs} = -\sum_j K_{ij}|^{rs} \cdot [D\,u|^{rs}]_j$, that is, $K_{ij}|^{rs}$ are constants. The homogeneous case is a particular one, provided that $K_{ij}|^{rs} = K_{ij}$ and isotropy is recovered when $K_{ij} = k\delta_{ij}$.
- Finally the function $K_{ij}|^{rs} = K_{ij}|^{rs}(u|^{11}, \dots, u|^{n_x n_y})$ may be parametrized using model parameters λ with physical meaning.

To facilitate the discussion, we shall analyze separately the effect of including heterogeneity/anisotropy and nonlinearities, since each problem has its own particularities, even if they may be simultaneously studied in one stroke. The ground truth solution is computed by the Method of Manufactured Solutions (Roache, 2001), that is, we impose the value of the field u and we compute the source term f, according to the particularities of each problem.

4.2. Heterogeneous and anisotropic problem

In this case, we seek for solutions to the problem given by Eqs. (13) for an heterogeneous problem (where $\boldsymbol{K} = k\mathbb{I}$, $k = k(x, y)$) and an anisotropic problem, where the diffusion is a second order symmetric tensor, such that

$$\boldsymbol{K} = \begin{pmatrix} K_{xx} & K_{xy} \\ K_{yx} & K_{yy} \end{pmatrix}, \tag{16}$$

where all the elements of the matrix $K_{ij} = K_{ij}(x, y)$.

4.2.1. Details about the method

Data generation. N profiles of the field $u(x, y; a, b, c)$ were synthetically generated, by sampling solutions of a parametric family, obtained for different values of $a, b, c \sim \mathcal{U}[0, 1]$ randomly and independently generated. Then, from this solutions, boundary conditions are assigned: $g_1 = u(x = 0, y)$, $g_2 = u(x = 1, y)$, $g_3 = u(x, y = 0)$ and $g_4 = u(x, y = 1)$. The same procedure is followed for the flux $q_x(x, y), q_y(x, y)$, where $q_1 = q(x = 0, y)$, $q_2 = q(x = 1, y)$, $q_3 = q(x, y = 0)$ and $q_4 = q(x, y = 1)$. These boundary conditions (from solution and flux) were considered as input variables to ensure that the PGNNIV is associated with a well-posed problem: we need at least one value of the flux, since the solution of Eqs. (13) is unique up to an additive constant value. The values of the field u were particularized at $n_x \times n_y$ points where $n_x = n_y = n = 10$, $x_i = y_i = \frac{i-1}{n-1}$, $i = 1, \dots, n$ so the output variables correspond therefore to the nodal values $u|^{ij} = u(x_i, y_j)$, $i, j = 1, \dots, 10$.

The equations for the field u, diffusivity $\boldsymbol{K}$, flux $\boldsymbol{q}$, and source term of the equation f for each of the chosen problems are shown below:

H: Heterogeneous problem

$$u(x, y) = a + b\ x + c\ y, \tag{17a}$$

$$k(x, y) = xy, \tag{17b}$$

$$q(x, y) = \begin{pmatrix} b\ xy \\ c\ xy \end{pmatrix}, \tag{17c}$$

$$f(x, y) = -(b\ y + c\ x). \tag{17d}$$

A: Anisotropic problem

$$u(x, y) = a + b\ x + c\ y + xy \tag{18a}$$

$$\boldsymbol{K}(x, y) = \begin{pmatrix} x + y + 1 & xy \\ xy & x - y + 1 \end{pmatrix}, \tag{18b}$$

$$q(x, y) = \begin{pmatrix} (x + y + 1)(b + y) + (c + x)\ xy \\ (b + y)\ xy + (x - y + 1)(c + x) \end{pmatrix}, \tag{18c}$$

$$f(x, y) = -(b + y + c\ y + 4xy + b\ x - c - x). \tag{18d}$$

Construction of the predictive network. Let us denote the tensor input shape as $\mathtt{x}$ (shape $[8, N]$), such that $\mathtt{x}[r, j] = g_r^j$, $\mathtt{x}[r, j] = q_{(r-4)}^j$, $r = 1, \dots, 4$; while the tensor output is denoted as $\mathtt{u}$ (shape $[n_x, n_y, N]$), such that $\mathtt{u}[i, j, k] = u_{ij}^k$. To predict the values of $\mathtt{u}$ we use standard ANN regression techniques. In this work, we consider MLP as the fundamental tool for building predictive models, although other neural network structures, such as autoencoders, may be suitable for other situations (Ayensa-Jiménez et al., 2024). In particular we use a 4-layer network, with two hidden layers (whose number of neurons are indicated in Table 1), such that $\mathtt{y} = \mathtt{Y}[\mathtt{x}]$ with $\mathtt{Y}$ the nonlinear operator that identifies the input–output relation in the neural network. With all these notations, the prediction error is

$$\mathtt{e} = \mathtt{y} - \mathtt{u}. \tag{19}$$

Construction of the continuum PGNNIV. We establish a new tensor operator $\mathtt{D}^*$, a first order finite difference filter associated with the first-order differential operator D, such that $\mathtt{dy}_x = \mathtt{D}_x^*[\mathtt{y}]$ (shape $[n_x - 1, n_y, N]$) and $\mathtt{dy}_y = \mathtt{D}_y^*[\mathtt{y}]$ (shape $[n_x, n_y - 1, N]$):

$$D_x^* = \frac{1}{L_x}\begin{pmatrix} 1 & -1 \\ 1 & -1 \end{pmatrix}, \qquad D_y^* = \frac{1}{L_y}\begin{pmatrix} 1 & 1 \\ -1 & -1 \end{pmatrix}, \tag{20}$$

where L_x and L_y are the mesh grid in x and y directions. Now we set a variable tensor $\mathtt{K}$, of shape $[n_x - 1, n_y - 1]$ and we define $\mathtt{q}_x = -\mathtt{dy}_x \cdot \mathtt{K}$, $\mathtt{q}_y = -\mathtt{dy}_y \cdot \mathtt{K}$. In this way and as the operators have been defined, the dimensions would not match, since, for example in the case of $\mathtt{q}_x$, we would have $\mathtt{K}$ of shape $[n_x - 1, n_y - 1]$ and $\mathtt{dy}_x$, of shape $[n_x - 1, n_y, N]$. This is the reason for introducing a last operator, $\mathtt{M}^*$, computing the mean between neighboring nodes in one of the two directions. We define $\mathtt{M}_x^*$ for the mean in the x axis such that for the field $\mathtt{dy}_y$, $\mathtt{M}_x^*[\mathtt{dy}_y]$ returns a field of shape $[n_x - 1, n_y - 1, N]$ and for the field $\mathtt{dy}_x$, $\mathtt{M}_y^*[\mathtt{dy}_x]$ returns a field of shape $[n_x - 1, n_y - 1, N]$. Thus, we define $\mathtt{q}_x = -\mathtt{M}_y^*[\mathtt{dy}_x] \cdot \mathtt{K}$ and $\mathtt{q}_y = -\mathtt{M}_x^*[\mathtt{dy}_y] \cdot \mathtt{K}$, both of shape $[n_x - 1, n_y - 1, N]$.

Finally, we define

$$\mathtt{f} = \mathtt{M}_y^*[\mathtt{D}_x^*[\mathtt{q}_x]] + \mathtt{M}_x^*[\mathtt{D}_x^*[\mathtt{q}_y]],$$

$$\tilde{\mathtt{u}} = (\mathtt{y}[0, \cdot, \cdot] - g_1, \mathtt{y}[n_x, \cdot, \cdot] - g_2, \mathtt{y}[\cdot, 0, \cdot] - g_3, \mathtt{y}[\cdot, n_y, \cdot] - g_4)$$

and

$$\tilde{\mathtt{q}} = (\mathtt{q}[0, \cdot, \cdot] - q_1, \mathtt{q}[n_x, \cdot, \cdot] - q_2, \mathtt{q}[\cdot, 0, \cdot] - q_3, \mathtt{q}[\cdot, n_y, \cdot] - q_4).$$

Consequently, the penalties involved in the problem are

$$\pi_1 = \mathtt{f}, \quad \pi_2 = \tilde{\mathtt{u}}, \quad \pi_3 = \tilde{\mathtt{q}}.$$

Table 1
PGNNIV hyperparameters for the linear problems.

Parameter		Value
Learning rate (first training)		3×10^{-4}
Learning rate (second training)		3×10^{-5}
Error coefficients (c_0)		10^7
Penalty coefficient	Flux conservation (c_1) Essentially boundary conditions (c_2) Natural boundary conditions (c_3)	10^4 10^3 10^5
Neurons in each hidden layer	H A	120 120
Parameters initialization	Weights Biases Explanatory operator	Xavier (Glorot and Bengio, 2010) Zeros Random normal
Total trainable parameters	H A	36 421 28 543
Batch size		64

Cost function, learning algorithm and hyperparameters. A PGNNIV may be interpreted as a standard ANN where the output space is augmented by including new variables (associated with the added constraints) whose exact value is identically zero (Ayensa-Jiménez et al., 2021). Therefore, as in all ANN problems, we have to specify a cost function and a learning algorithm and its associated hyperparameters, but also we have to specify the weights associated with the PILs related to the constraints. In the formal minimization problem, these weights are the penalty coefficients that become, therefore, new hyperparameters. In this work, we consider the mean squared error (MSE) as the cost function, both for the error and penalty terms, and we select Adaptive Moment (Adam) optimizer (Kingma and Ba, 2014). Note that when referring to the MSE of a tensor, we understand the MSE as the sum of the squares of all its components. The differences in the tensor sizes and physical nature (i.e. units) is the reason for the capital role of the selection of the penalty weights. As training methodology, we first apply a higher learning rate to achieve rapid convergence toward an approximate optimal solution. Then, the learning rate is reduced to refine the model accuracy and minimize the training error. The resulting loss function $\mathcal{L}$ of the optimization procedure is, therefore:

$$\mathcal{L} = c_0 \mathrm{MSE}(\mathrm{e}) + \sum_{i=1}^{3} c_i \mathrm{MSE}(\pi_i). \tag{21}$$

The network architecture and hyperparameters of the PGNNIV are summarized in Table 1. To evaluate the performance of the continuum-based PGNNIV we generated $N = 10^4$ samples of input–output values for problems H and A. We used 80% of the generated values as training data and 20% as test data. At each iteration along the optimization process, the PGNNIV was fed with the whole training data. The first iterative process was stopped after 10^4 iterations, and the second iterative process with a lower learning rate, was terminated after 10^5 iterations, to balance computational efficiency with model optimization.

4.2.2. Results

Network convergence. The convergence of neural networks is shown in Fig. 4, where a moving average filter has been used to smooth curves reducing noise and highlighting trends, and in Fig. 5, where normalized loss for each penalty is shown. Normalized loss was computed by normalizing all values relative to the maximum loss obtained.

Predictive capacity. Once the network has converged, we can predict the values of the field u by simply interpolating the obtained values for the nodes, $u|^{ij}$, predicted by the network for any value of the boundary conditions. Note that the computation of the nodal values has a minimal cost (the one of a single ANN evaluation), since it does not require the inversion of any system of equations neither any iteration procedure, and to compute the field at any point we only add the cost of an interpolation. Moreover, the values of the fields $\boldsymbol{q}$ and k are obtained

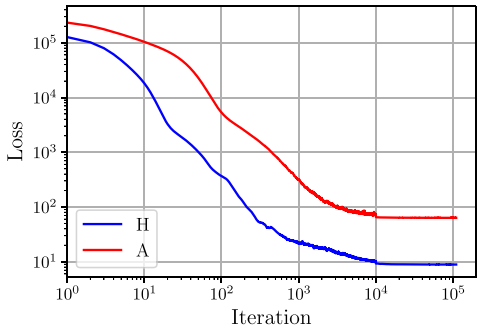
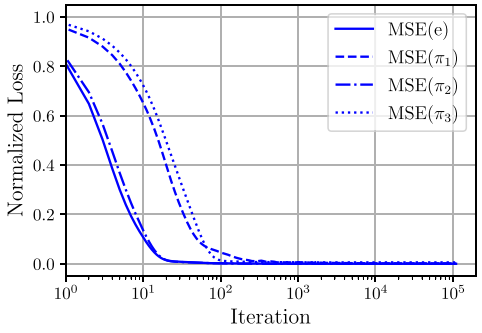
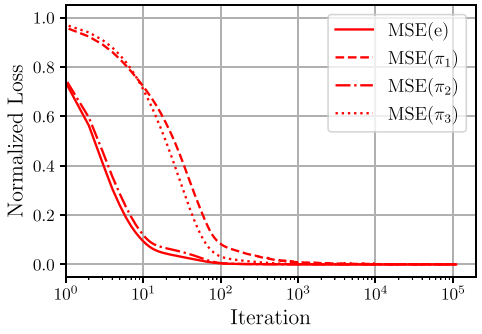


Fig. 4. Network convergence. Learning curve for the heterogeneous problem (H) and the anisotropic problem (A). Curves have been filtered with moving average filter using a convolution with a uniform window.

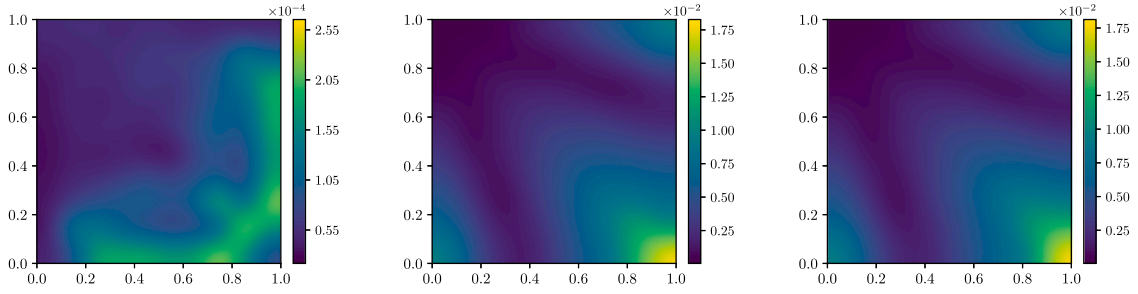


(a) Heterogeneous problem (H).



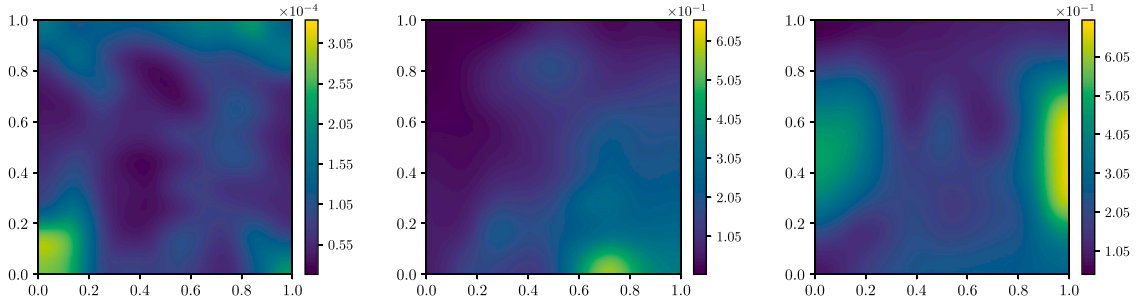
(b) Anisotropic problem (A).

Fig. 5. Evolution of the loss term along the optimization process. The different loss terms are normalized by its maximal value for a simpler and more consistent comparison.



(a) Solution $u(x, y)$. (b) Flux $q_x(x, y)$. (c) Flux $q_y(x, y)$.

Fig. 6. Mean Absolute Difference (MAD) between real and predicted variables in the heterogeneous problem. A bicubic interpolator has been used to interpolate for the obtention of smooth plots.



(a) Solution $u(x, y)$. (b) Flux $q_x(x, y)$. (c) Flux $q_y(x, y)$.

Fig. 7. Mean Absolute Difference (MAD) between real and predicted variables in the anisotropic problem. A bicubic interpolator has been used to interpolate for the obtention of smooth plots.

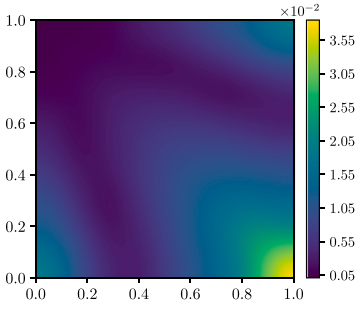


Fig. 8. Mean absolute difference between real and predicted diffusivity in the heterogeneous problem. A bicubic interpolator has been used to interpolate for the obtention of smooth plots.

as a byproduct of the network without any post-process beyond the nodal interpolation. Figs. 6 and 7 show the mean absolute difference between the real values and the predicted values for the solution u and the flux $\boldsymbol{q}$ in direction x and y

$$\epsilon_f(x, y) = \frac{1}{N}\sum_{i=1}^{N} |\hat{f}(x, y, a_i, b_i, c_i) - f(x, y, a_i, b_i, c_i)|,$$

with $\hat{f}$ being the predicted value of the field and f the exact value and $f = u, q_x, q_y$.

Additionally, the statistics of the normalized L_2 errors corresponding to the prediction of the different fields have been obtained and are shown in Tables 2–4 for the test dataset. This error is computed by using the estimate

$$E_r[f] = \sqrt{\frac{\int_0^1\int_0^1 (f(x, y) - \hat{f}(x, y))^2 \,\mathrm{d}x\,\mathrm{d}y}{\int_0^1\int_0^1 (f(x, y))^2 \,\mathrm{d}x\,\mathrm{d}y}}, \tag{22}$$

Table 2
Statistics of the error for the solution u for the two different problems.

	$E_r[u]$				
	Min	Q1	Q2	Q3	Max
H	3.31×10^{-5}	7.79×10^{-5}	1.06×10^{-4}	1.46×10^{-4}	1.07×10^{-3}
A	1.83×10^{-5}	5.87×10^{-5}	8.43×10^{-5}	1.17×10^{-4}	1.16×10^{-3}

Table 3
Statistics of the error for the flux q_x for the two different problems.

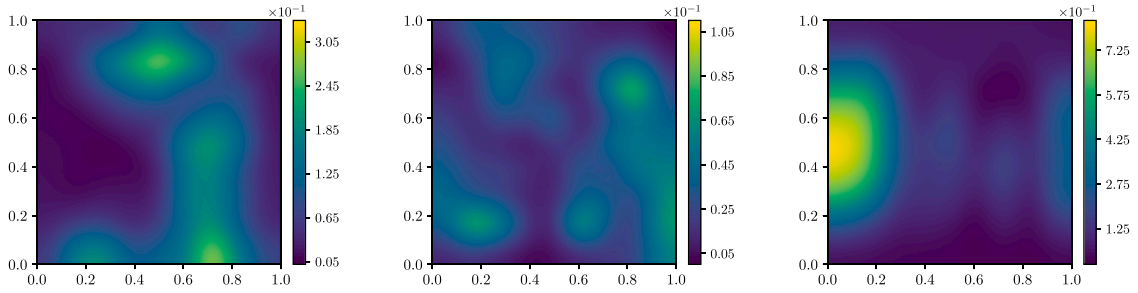
	$E_r[q_x]$				
	Min	Q1	Q2	Q3	Max
H	2.61×10^{-2}	2.80×10^{-2}	2.81×10^{-2}	2.83×10^{-2}	2.15
A	5.30×10^{-2}	6.01×10^{-2}	6.98×10^{-2}	8.69×10^{-2}	1.69×10^{-1}

Table 4
Statistics of the error for the flux q_y for the two different problems.

	$E_r[q_y]$				
	Min	Q1	Q2	Q3	Max
H	2.74×10^{-2}	2.80×10^{-2}	2.81×10^{-2}	2.84×10^{-2}	1.62×10^{1}
A	1.53×10^{-1}	1.60×10^{-1}	1.75×10^{-1}	2.00×10^{-1}	2.77×10^{-1}

which is indeed a random variable as it depends on the sample $i = 1, \ldots, N$. Note that, except for some very particular predictions, the error remains small. Also, the errors associated to the flux field $\boldsymbol{q}$ are larger, as this is the non-measurable variable and therefore the one that is not directly supervised. In Appendix A, the predictions of u and $\boldsymbol{q}$ fields for some sampled cases are shown, as well as more details about the distribution of the errors are depicted.

Unraveling capacity. As explained in Section 4.1, PGNNIVs have both predictive and unraveling capacity. This has been explored in the precedent example by reproducing the field $k = k(x, y)$. This field is a direct



(a) K_{xx}. (b) $K_{xy} = K_{yx}$. (c) K_{yy}.

Fig. 9. Mean Absolute Difference (MAD) between real and predicted diffusivity tensor for its different components in the anisotropic problem. A bicubic interpolator has been used to interpolate for the obtention of smooth plots.

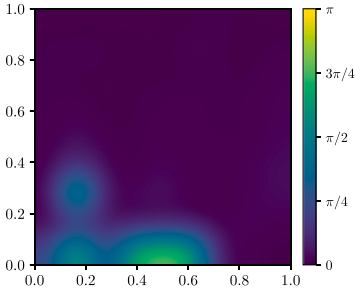


Fig. 10. Difference in angles between the two principal directions of diffusion. A bicubic interpolator has been used to interpolate for the obtention of smooth plots.

Table 5
Error for the diffusivity k for the two different problems.

	$E_r[\boldsymbol{K}]$
H	2.82×10^{-2}
A	1.25×10^{-1}

output of the problem, but when expressed in terms of the variables x, y, it may be seen as an explanation (identification) of the constitutive model. In the case of the heterogeneous problem, the field $k(x, y)$ is computed using a filter of shape $[n_x, n_y, 1]$, and, in the case of the anisotropic problem, as a tensor of size $[n_x, n_y, 2, 2]$ with second order symmetry at the last two indexes, that is introduced in the constitutive relation, Eq. (13b). As the output field $k(x, y)$ does not depend on the value of the boundary conditions, all quantile indicators collapse to a single error value, as shown in Table 5. Moreover, Fig. 8 shows the (mean) absolute difference in each node between the real values and the predicted values for the diffusivity k for the heterogeneous problem, and Fig. 9 shows the same magnitude for the three components of the diffusivity tensor $\boldsymbol{K}$ in the anisotropic problem.

The unraveling capacity becomes even more interesting in the case of the anisotropic problem, in which the diffusivity is a tensor, and the behavior of materials can be obtained through the properties of the diffusion tensor. For instance, the microstructure of the material may be unraveled by exploring the direction of maximal diffusivity, that is associated with some directional character of its microstructure. Assuming that Eqs. (18) represent perfectly the diffusion process for the considered material, we compute the difference between the predicted and actual result in angles between the direction of maximum diffusivity with respect to the x-axis. We observe low errors for almost all the domain, except close to the boundaries where the local behavior is obviously not so well captured, as seen in Fig. 10.

In Appendix A, the prediction of the diffusivity field and the comparison between the predicted and learned diffusivity function are shown as well as the comparison between the first principal direction of the predicted diffusion tensor when compared to ground truth.

4.3. Nonlinear problem

Now we explore two solutions to a nonlinear version of the problem given by Eq. (13), by making $k = k(u) = u$ and $k = k(u) = u(1 - u)$.

4.3.1. Details about the method

Data generation. We define the following expressions of the field u, diffusivity k, flux $\boldsymbol{q}$, and source term of the equation f, that satisfy identically Eq. (13).

NL1: Nonlinear Problem 1

$$u(x, y) = \sqrt{a + bx + cy}, \tag{23a}$$

$$k(x, y) = k(u) = \sqrt{a + bx + cy}, \tag{23b}$$

$$q(x, y) = \begin{pmatrix} \frac{b}{2} \\ \frac{c}{2} \end{pmatrix}, \tag{23c}$$

$$f(x, y) = 0. \tag{23d}$$

NL2: Nonlinear Problem 2

$$u(x, y) = \sqrt{a + bx + cy}, \tag{24a}$$

$$k(x, y) = k(u) = (1 - \sqrt{a + bx + cy})\sqrt{a + bx + cy}, \tag{24b}$$

$$q(x, y) = \begin{pmatrix} \frac{b}{2}(1 - \sqrt{a + bx + cy}) \\ \frac{c}{2}(1 - \sqrt{a + bx + cy}) \end{pmatrix}, \tag{24c}$$

$$f(x, y) = \frac{1}{4\sqrt{a + bx + cy}} \left(b^2 + c^2\right). \tag{24d}$$

The input and output values that feed the predictive neural network are generated analogously to the previous example, in terms of the parameters a, b and c, generated again using independent uniform random distributions.

Construction of the predictive network. One of the advantages of this methodology is that the predictive neural network only depends on the nature of the input and output variables. Therefore, the predictive network used for this nonlinear problem is similar to the one used in the previous example, but with 150 neurons in each layer as described in Table 6. Indeed, the nature of the hidden state equation only affects the physical constraints associated with the PILs layers and therefore the explanatory network.

Construction of the continuum PGNNIV. From the tensor `y` we obtain the derivative field in directions x and y using again the tensor operator `D`*. However, we define additionally a new tensor `k` representing the diffusivity associated with each element. Note that this field has a shape of $[n_x - 1, n_y - 1, N]$ as it is defined on the elements rather than on the nodes. When linking the diffusivity with the value of the field, this has to be done at the element level, so we define an element field

Table 6
PGNNIV hyperparameters for the nonlinear problems.

Parameter		Value
Learning rate (first training)		3×10^{-4}
Learning rate (second training)		3×10^{-5}
Error coefficients (c_0)		10^7
Penalty coefficient	Flux conservation (c_1) Essentially boundary conditions (c_2) Natural boundary conditions (c_3)	10^4 10^3 10^5
Neurons in each layer	Predictive network Explanatory network	150 150
Number of filters		5
Parameters initialization	Weights Biases	Xavier Zeros
Total trainable parameters	NL1 NL2	51 571 51 571
Batch size		64

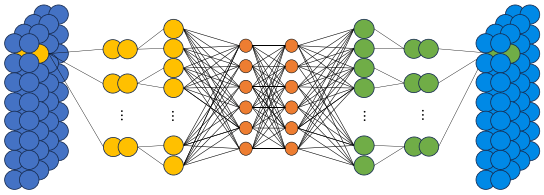


Fig. 11. Moving Multilayer Perceptron (mMLP) scheme used for the explanatory network. The values of the different architecture hyperparameters are the ones reported in Table 6.

$\texttt{um} = \texttt{M}_y^*[\texttt{M}_x^*[u]]$ (shape $[n_x - 1, n_y - 1, N]$) obtained by averaging the nodal values of the field associated with the considered element. The next step is to define an ANN relating the tensors um and k. Note that the point-wise character of this (unknown) relationship, that is, the fact that $\texttt{k}[i, j, N] = f(\texttt{um}[i, j, N])$, is easily formulated in a DL framework by defining a CNN that acts as a mMLP, represented in Fig. 11. We expand each neuron value using m filters, that are connected again to a 4-layer MLP with two hidden layers (whose number of neurons in each layer is reported in Table 6). This subnetwork expands in a higher dimensional space the content of each neuron associated with each element to capture the diffusion nonlinearity. For more details the code is available so the technical and implementation details may be consulted.

Cost function, learning algorithm and hyperparameters. All algorithms and hyperparameters used in this example are the same as in the preceding one, with the exception that as mentioned above, there is an extra hyperparameter related to the network architecture, which is the number of applied filters of the mMLP in the explanatory network, that is, m. For completeness, they are reported in Table 6.

4.3.2. Results

Network convergence. The convergence in terms of the loss function and the different penalty terms presents the same trend discussed before, as observed in Fig. 12 and Fig. 13 respectively.

Predictive capacity. As in the previous examples, the performance of the presented methodology for predicting the value of the different fields is now evaluated. There is, however, a difference with respect to previous cases that is inherent to nonlinear problems: the value of the field $k(x, y)$ depends now on the boundary conditions (different for each data sample), as $k = k(u(x, y))$ and u depends on the boundary conditions. Fig. 14 and Fig. 15 show the mean absolute difference in each node between the real values and the predicted values for the solution and the flux in the x and y direction. Fig. 16 and Fig. 17 show the same magnitude but for $k(x, y)$.

Moreover, the statistics of the errors for the boundary conditions varying in the whole learning space are shown in Tables 7, 8, 9 and 10 for the fields u and $\boldsymbol{q}$ and for the diffusivity field k, respectively.

In Appendix A, we present the predictions of the fields u, $\boldsymbol{q}$ and k for some sampled cases, as well as the distribution of the errors.

Table 7
Statistics of the error for the solution u for the nonlinear problem.

	$E_r[u]$				
	Min	Q1	Q2	Q3	Max
NL1	6.20×10^{-6}	1.99×10^{-5}	2.96×10^{-5}	4.04×10^{-5}	9.27×10^{-4}
NL2	4.14×10^{-5}	4.71×10^{-5}	5.11×10^{-5}	5.99×10^{-5}	1.33×10^{-3}

Table 8
Statistics of the error for the flux q_x for the nonlinear problem.

	$E_r[q_x]$				
	Min	Q1	Q2	Q3	Max
NL1	1.85×10^{-4}	3.85×10^{-4}	5.43×10^{-4}	1.11×10^{-3}	1.06
NL2	1.06×10^{-2}	2.05×10^{-2}	4.64×10^{-2}	5.38×10^{-2}	1.24

Table 9
Statistics of the error for the flux q_y for the nonlinear problem.

	$E_r[q_y]$				
	Min	Q1	Q2	Q3	Max
NL1	2.08×10^{-4}	3.94×10^{-4}	5.65×10^{-4}	1.09×10^{-3}	1.94
NL2	1.15×10^{-2}	2.14×10^{-2}	4.64×10^{-2}	5.39×10^{-2}	7.09

Table 10
Statistics of the error for the diffusivity k for the nonlinear problem.

	$E_r[k]$				
	Min	Q1	Q2	Q3	Max
NL1	4.20×10^{-4}	6.09×10^{-4}	8.57×10^{-4}	1.30×10^{-3}	9.46×10^{-3}
NL2	9.69×10^{-3}	1.79×10^{-2}	4.63×10^{-2}	5.87×10^{-2}	3.09×10^{-1}

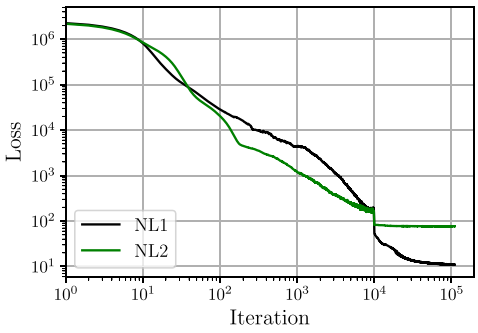
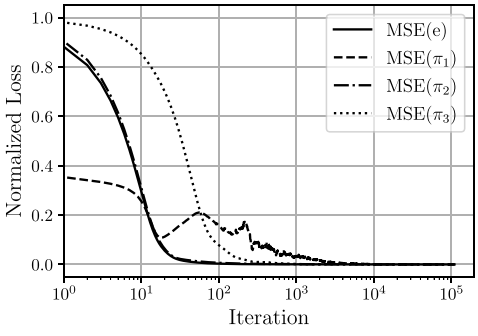
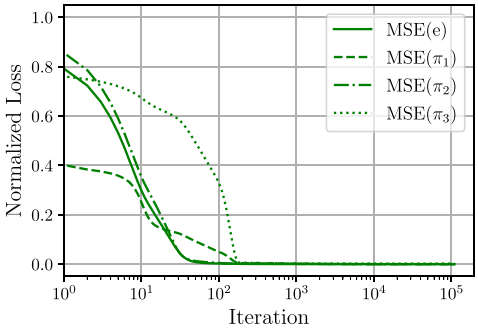


Fig. 12. Network convergence. Learning curve for both non lineal problems (NL1 and NL2). Curves have been filtered with moving average filter using a convolution with a uniform window.



(a) Non lineal problem NL1 penalties



(b) Non lineal problem NL2 penalties

Fig. 13. Evolution of the loss term along the optimization process. The different loss terms are normalized by its maximal value for a simpler and more consistent comparison.

Unraveling capacity. As explained before, PGNNIVs have both predictive and unraveling capacity. For nonlinear problems, however, this capacity is exploited more satisfactorily. The interest here is to learn the model $k = k(u)$. As well as for predictions, in Appendix A the prediction of the diffusivity field and the comparison between the predicted and learned diffusivity function is presented.

To evaluate the explanatory capacity of the PGNNIV, we evaluate the error in the discovery of the constitutive relation $k = k(u)$, that is, the ability of the method to unravel the relationship between the diffusion coefficient and the solution field u. To do so, we define the following explanatory error functional

$$\mathcal{E}_r = \sqrt{\frac{\sum_{i=1}^{N}\sum_{j=1}^{n_x}\sum_{k=1}^{n_y}\left(\hat{k}^i\left(x_j,y_k\right) - k\left(u^i\left(x_j,y_k\right)\right)\right)^2}{\sum_{i=1}^{N}\sum_{j=1}^{n_x}\sum_{k=1}^{n_y}\left(k\left(u^i\left(x_j,y_k\right)\right)\right)^2}} \tag{25}$$

where $\hat{k}^i(x,y)$ is the learned diffusion at point (x,y). Indeed, with Eq. (25) we compute the (relative) error between the predicted and actual values of the function k along space and samples. The value of the explanatory error for the presented problem is reported in Table 11.

Table 11
Relative error of the unraveling capacity of the PGNNIV for the nonlinear problem.

	$\mathcal{E}_r$
NL1	6.84×10^{-4}
NL2	3.28×10^{-2}

5. Discussion

The PGNNIV framework introduces a new and singular way of combining physical knowledge and ML techniques to solve problems in continuum physics, here illustrated with the simple yet illustrative stationary diffusion equation. Indeed, even if the presented case studies are mathematically simple and academic, they highlight all ingredients and main features of the methodology, as have been enriched with an extra complexity such as heterogeneity, anisotropy and nonlinearity. Thus, the presented work illustrates the capacity of the method to deal with arbitrary complex models, equations and structures. We have considered nonlinearities and different degrees of material knowledge such as spatial and rotational symmetries. Also, locality has been implicitly considered when establishing a local relation between the flow $\boldsymbol{q}$ and the essential field $\boldsymbol{u}$. In addition, the flexibility to add some or the whole available physical knowledge to the network has been exploited: several degrees of knowledge have been tested, involving the aforementioned symmetries and the mathematical character of the constitutive operators. In that sense, the presented methodology can be seen as a tool for distilling knowledge from data. Indeed, the data-information-knowledge flow can be tracked as follows:

1. **Data**: There are the initial numerical values $\mathcal{D} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i), i = 1, \dots, N\}$. These input and output vectors internally contain all the physical information, but not in an easily identifiable way.
2. **Information**: The hidden relations between input and output data are encoded in the predictive network Y, via its structure and its associated parameters. Indeed, by knowing the network architecture and the value of the parameters it is possible (in principle) to reproduce the map $\boldsymbol{x} \mapsto \boldsymbol{y}$, that contains the relevant system information. A special case is when the predictive network has the structure of an autoencoder. In that case, the latent space will contain the minimal information contained in the input data for accurately predicting the system response (Ayensa-Jiménez et al., 2024).
3. **Knowledge**: The knowledge about the system is encoded in the explanatory network H. This is actually the network that contains the most high level information in the sense that it can be easily interpreted and comprehended and give us the most distilled information about the system.

In addition, features such as spatial and rotational symmetries, as well as material nonlinearities are actually the macroscopic representation of some microstructural patterns. For instance, we have demonstrated when solving problem A that it is possible to unravel the directional microstructure of an anisotropic material from macroscopic measurements, provided that this anisotropy is translated to a macroscopic

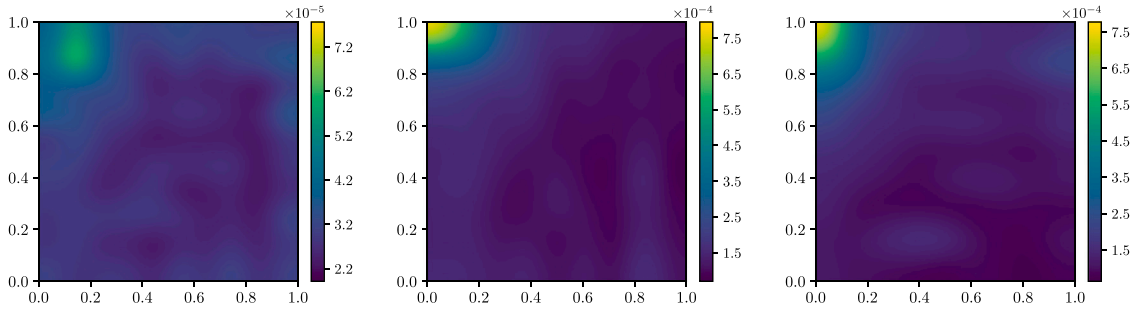


Fig. 14. Mean absolute difference between real and predicted variables for NL1. A bicubic interpolator has been used to interpolate for the obtention of smooth plots.

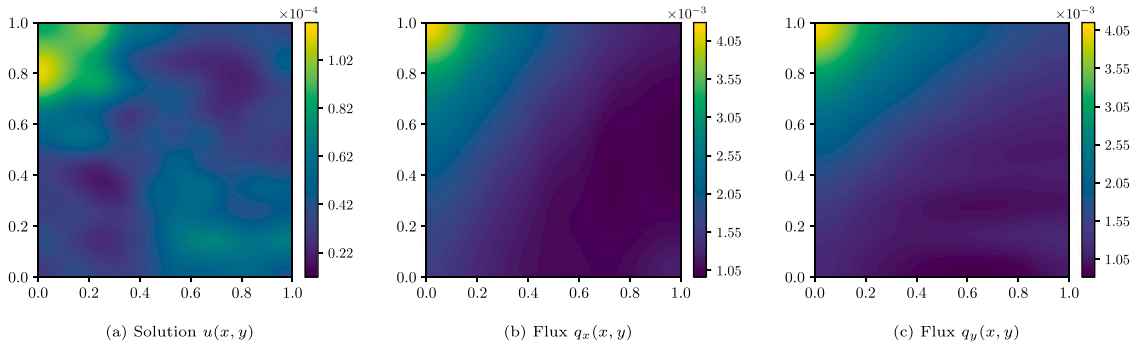


Fig. 15. Mean absolute difference between real and predicted variables for NL2. A bicubic interpolator has been used to interpolate for the obtention of smooth plots.

specific behavior, such as particle (or thermal) diffusivity. In that sense, the presented methodology is a tool for translating macroscopic to microscopic information.

Nonetheless, there is an important limitation of the method. It is crucial to correctly choose the best strategy when selecting what is known and what is not in the problem to solve. If something is known, the best strategy is to include it in the PGNNIV if possible, explicitly (strong form, by means of the network architecture) or implicitly (weak form, by means of the loss function). In the limit case, parametric models are the best ones and less expensive to train, so they give the best results, when correctly assumed, as has been demonstrated after centuries of material science modeling.

This may drive to think that in that limit case nothing new has been presented, but this is simplistic since the model learning and the predictive capacity are acquired in one stroke and once for all and can be continuously improved by new sets of data input, following the strategy of DDDAS (Blasch et al., 2022). Then, any prediction may be performed later via offline calculation in one single evaluation. This is extremely advantageous for optimization, inverse problems or stochastic computations based on Monte Carlo strategies, among others. In particular, we may set the whole constitutive model (that is, we can replace the explanatory network by the true constitutive model), which is equivalent to build a response surface using ANN procedures or to derive a surrogate model using back-propagation, with the particularity that the computational cost is reduced due to the constraints with respect to usual ANN procedures (Ayensa-Jiménez et al., 2021).

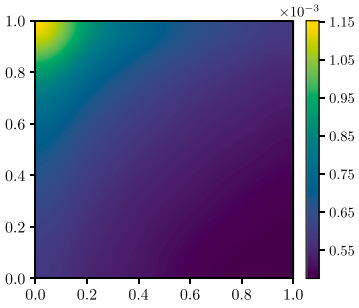


Fig. 16. Mean absolute difference between real and predicted k for NL1. A bicubic interpolator has been used to interpolate for the obtention of smooth plots.

Finally, two last important remarks should be done about the limitations of the presented methodology:

- The first one is the obvious fact that the methodology is based on the availability of enough data. Data quantity but mainly data quality is required: large amounts of data are not enough, but they have also to be well distributed for the model to be accurately learned. This is more complicated to be thought, since there is not a simple way to guarantee that internal variables have a pertinent coverage when the constitutive relation is not *a priori* known. However, since the data used corresponds always to measurable

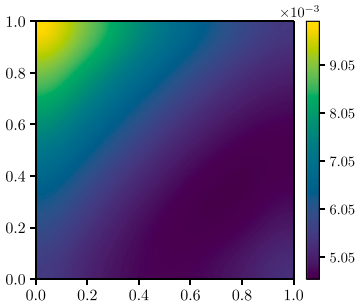


Fig. 17. Mean absolute difference between real and predicted diffusivities for NL2. A bicubic interpolator has been used to interpolate for the obtention of smooth plots.

data obtained in any non-specific situation, we can think of bunches of videos or sensor measurements that will probably be the future of external data feeding scientific networks.

- A rough increase of the discretization (that is, to augment the number of nodes or to reduce the mesh size) does not necessarily improve the prediction or unraveling capacity. This is a fundamental difference between this methodology and the usual simulation approach (that is merely predictive). Here we point out the need of new research results in this line, related to the mathematical structure of a broad range of problems. In a certain sense, we still suffer from the lack of mathematical results in ANN theory, playing the role that, for instance, convergence theorems play in Finite Element analysis.

6. Conclusion

In the present work, we introduced a general framework for the analysis of problems in continuum physics using elements of Machine Learning, incorporating the fundamental physical laws as well as material symmetries in the computations. We have taken advantage of the characteristics of a very recent idea, the so-called Physically-Guided Neural Networks with Internal Variables (PGNNIV), for a very general formulation of a broad class of problems whose physical content is expressed by means of a system of partial differential equations. The key point is a cunning splitting of the equations of the problem into two sets, what is known and what is not, and therefore it is intended to be learned, that is translated into the interaction between two networks, the predictive and the explanatory one. The method has been tested with a simple although illustrative case as it is the diffusion equation, incorporating complex features such as heterogeneity, anisotropy and nonlinearity.

It has been demonstrated that our approach offers both predictive and unraveling capacity: it is possible to predict in a single evaluation the state of the system for any prescribed value of the input variable, and to learn about the mathematical structure of the constitutive state equation of the problem. In addition, it is possible to infer information about the microstructure of a material from macroscopic data obtained without the need for specimen extraction neither homogeneous test design. Indeed, good results have been demonstrated for some paradigmatic cases in science and engineering related to constitutive modeling: local character, heterogeneity and nonlinearity.

The results are subordinated to the trends and features that are common to Machine Learning techniques, in particular, the importance of the dataset, the filtering capacity and the problem of overfitting. Nevertheless, the numerical experiments and examples shown in this work should be reinforced by further research exploring the mathematical structure of the PGNNIV problem, depending on the particular selection of the topology for the predictive and the explanatory networks, as well as the prescription of the input and output variables and the physical constraints. In any case, PGNNIV are very promising for both predicting and unraveling problems formulated in terms of partial differential equations, ubiquitous in sciences and engineering.

All computational experiments are provided in a GitHub repository, where the full source code of the work is made available for review and further exploration.

CRediT authorship contribution statement

Rubén Muñoz-Sierra: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis. **Jacobo Ayensa-Jiménez:** Writing – review & editing, Writing – original draft, Supervision, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Manuel Doblaré:** Writing – review & editing, Project administration, Methodology, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Manuel Doblare reports financial support was provided by Beonchip S.L. Manuel Doblare reports financial support was provided by Government of Aragón. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to acknowledge the financial support from Beonchip S.L. through the project DIAMOOC IDI-20220819, and the Government of Aragon, Spain (group DGA-T62_23R).

Appendix

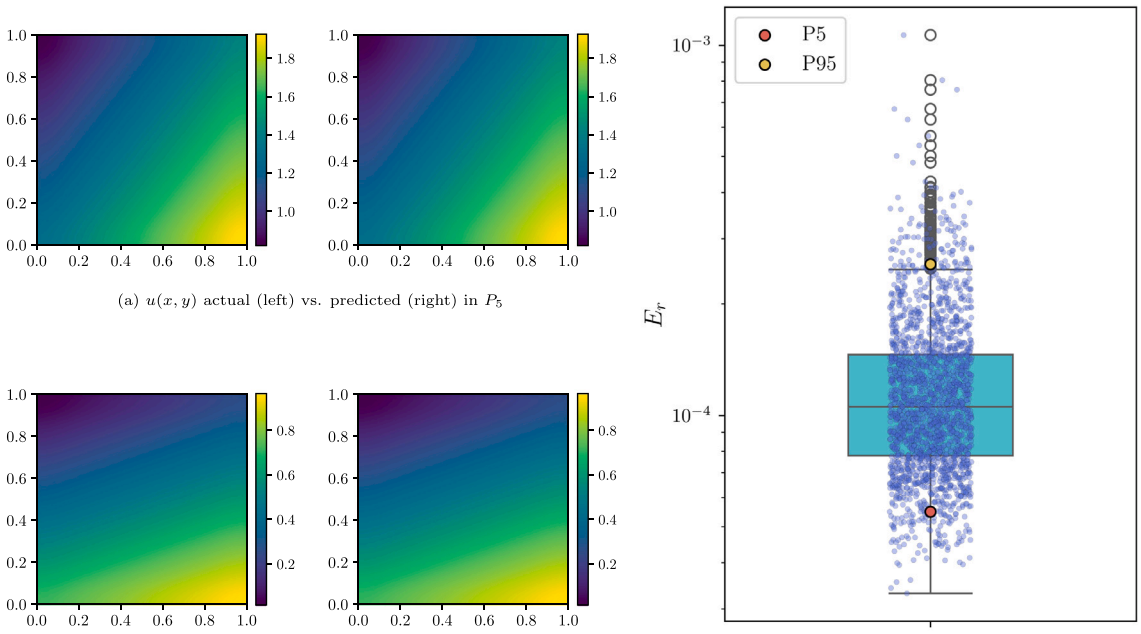
In the main text, we have presented different error plots comparing the predictions and actual values for different magnitudes and different proposed problems. In this appendices section, we provide detailed plots to illustrate the method predictive and explanatory capacity.

Appendix A. Details of results

A.1. Heterogeneous problem (H)

Predictive capacity. In Figs. 18, 19, and 20, a boxplot of the relative quadratic errors, Eq. (22), between the predictions and the real values is shown. Additionally, a prediction with an error value in the 5th percentile (P_5) and another in the 95th percentile (P_{95}) are selected for illustration purposes, being this compared with the real fields. This is done for the solution $u(x, y)$ in Fig. 18, as well as for the flux, q_x in Fig. 19 and q_y in Fig. 20. Fig. 21 illustrates the relative error as a function of these three parameters for the predicted values. The coordinate axes display the variables a, b, and c, while the color bar represents the absolute value of relative error E_r. As it is natural, the worst predictions are always close to the boundaries of the learning domain.

Explanatory capacity. Fig. 22 shows the comparison between the real diffusivity and the predicted diffusivity in problem H.



(a) $u(x, y)$ actual (left) vs. predicted (right) in P_5

(b) $u(x, y)$ actual (left) vs. predicted (right) in P_{95}

(c) Boxplot of relative error in predictions of u

Fig. 18. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for $u(x, y)$ in the heterogeneous problem.

A.2. Anisotropic problem (A)

Predictive capacity. As in the previous section, in Figs. 23, 24, and 25, a boxplot of the relative quadratic error between the predictions and the real values is shown as well as some illustrative samples.

Fig. 26 illustrates again the relative error as a function of the variables a, b, and c for the anisotropic problem showing again that the worst predictions are always close to the boundaries of the learning domain.

Explanatory capacity. In the case of the anisotropic problem, there are three independent components for the diffusion tensor, that are displayed in Fig. 27.

For this problem, the diffusion tensor encodes the information on the microstructural properties of the material. For instance, in that case, the principal diffusion directions of the tensor indicate the directions of maximal and minimal diffusion, that eventually correlate with the orientation of the microstructure. The error in the prediction of the angle was shown in Section 4.2.2, Fig. 9. We also represent them as in Fig. 28, where we compare the predictions with the real main microstructure direction.

A.3. Nonlinear problem (NL1)

Predictive capacity. As in the previous section, Figs. 29, 30, and 31, show a boxplot of the relative quadratic error between the predictions and the real values as well as some illustrative samples.

Fig. 32 illustrates again the relative error as a function of the variables a, b, and c for the nonlinear problem NL1 showing again that the worst predictions are always close to the boundaries of the learning domain.

Explanatory capacity. In this case, since $k = k(u)$ will be given by a neural network, referred to as the explanatory network, and will depend on the boundary conditions, there will be one prediction for each set of boundary conditions. These predictions can then be compared with the actual values as shown in Fig. 33.

Finally, Fig. 34 presents a comparison between the actual plot and the validation plot of $k(u)$ with respect to $u(x, y)$.

A.4. Nonlinear problem (NL2)

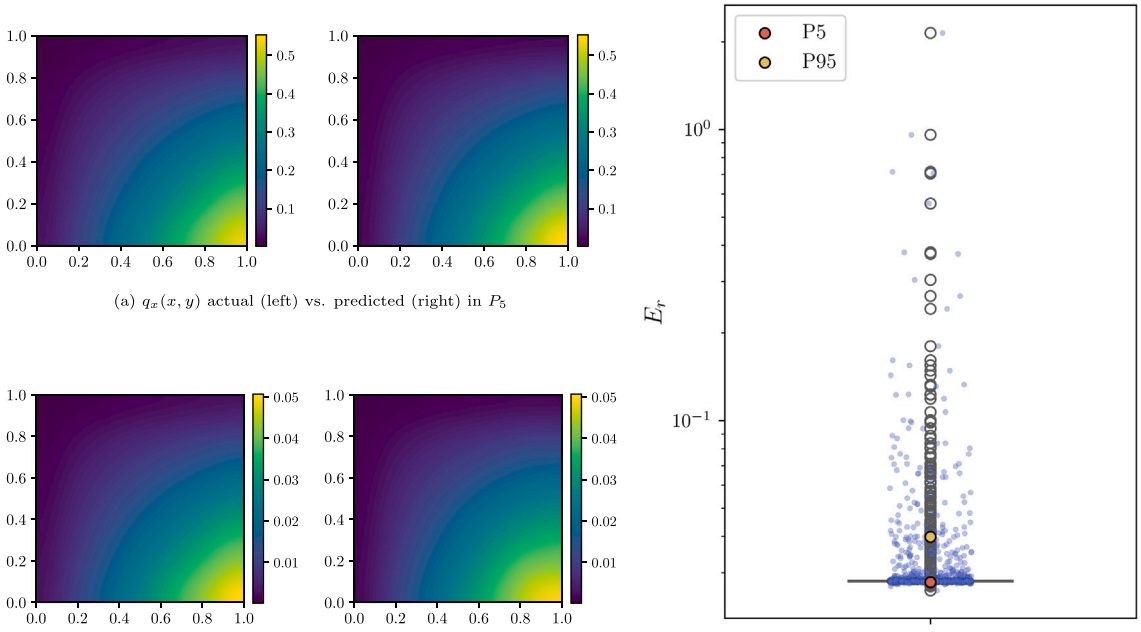
Predictive capacity. As in the previous section, Figs. 35, 36, and 37, show a boxplot of the relative quadratic error between the predictions and the real values as well as some illustrative samples. Fig. 38 illustrates again the relative error as a function of the variables a, b, and c for the nonlinear problem NL2 showing again that the worst predictions are always close to the boundaries of the learning domain.

Explanatory capacity. In this case, since $k = k(u)$ will be given by a neural network, referred to as the explanatory network, and will depend on the boundary conditions, there will be one prediction for each set of boundary conditions. These predictions can then be compared with the actual values as shown in Fig. 39.

Finally, Fig. 40 presents a comparison between the actual plot and the validation plot of $k(u)$ with respect to $u(x, y)$.

Data availability

Software and data are available in the following repository: https://github.com/rmunozTMELab/PGNNIV-to-Continuum-Problems.git.

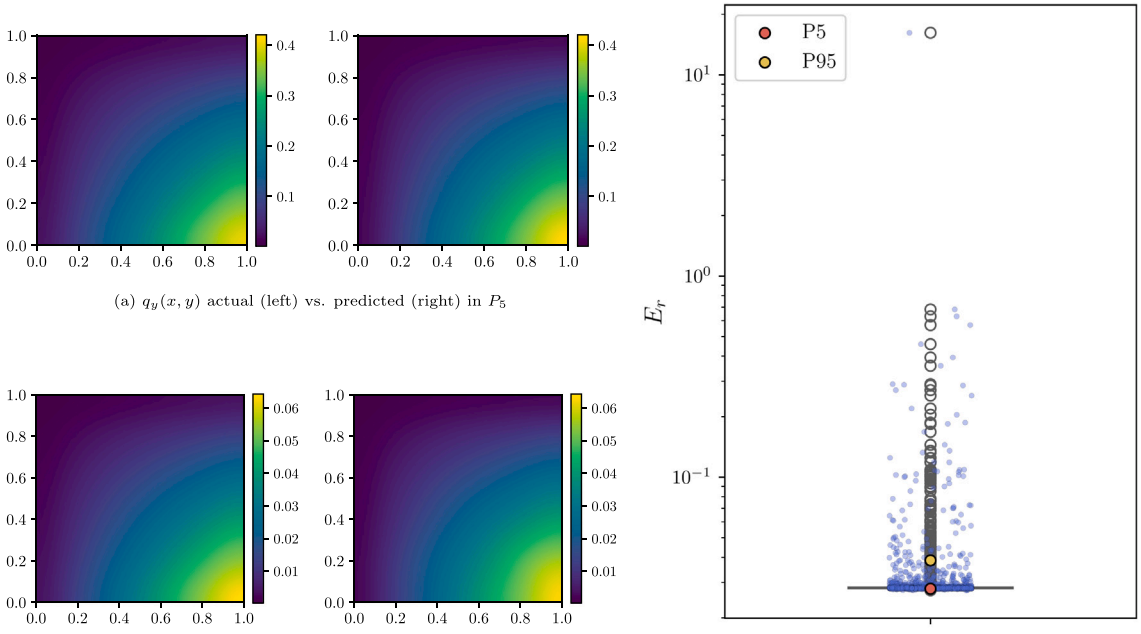


(a) $q_x(x, y)$ actual (left) vs. predicted (right) in P_5

(b) $q_x(x, y)$ actual (left) vs. predicted (right) in P_{95}

(c) Boxplot of relative error in predictions of q_x

Fig. 19. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for $q_x(x, y)$ in the heterogeneous problem.

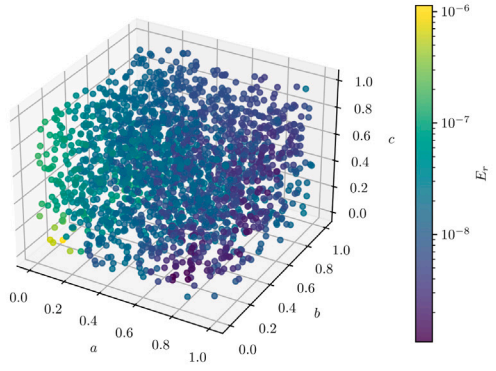


(a) $q_y(x, y)$ actual (left) vs. predicted (right) in P_5

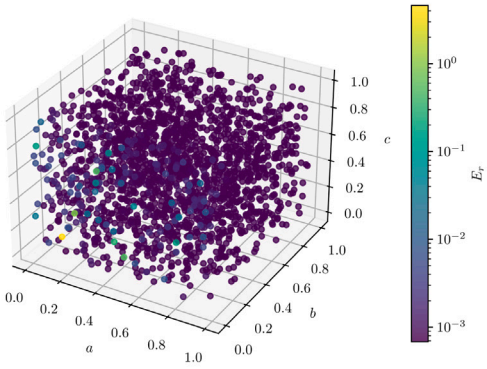
(b) $q_y(x, y)$ actual (left) vs. predicted (right) in P_{95}

(c) Boxplot of relative error in predictions of $q_y(x, y)$

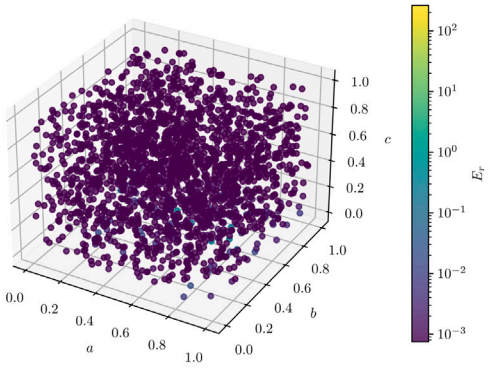
Fig. 20. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for $q_y(x, y)$ in the heterogeneous problem.



(a) E_r of predicted solutions $u(x, y)$ for parameters a, b, and c.



(b) E_r of predicted flux $q_x(x, y)$ for parameters a, b, and c.



(c) E_r of predicted flux $q_y(x, y)$ for parameters a, b, and c.

Fig. 21. E_r of predicted variables for parameters a, b, and c in the heterogeneous problem.

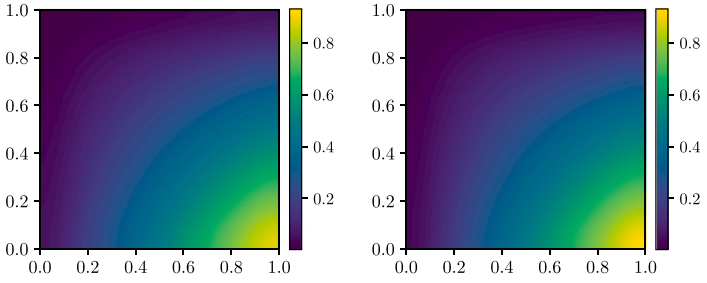
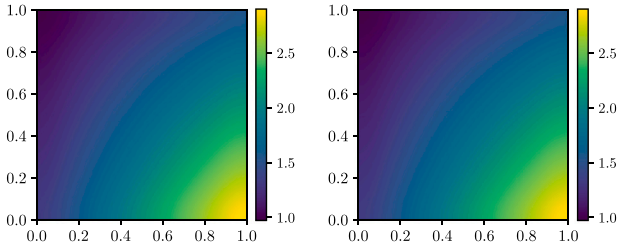
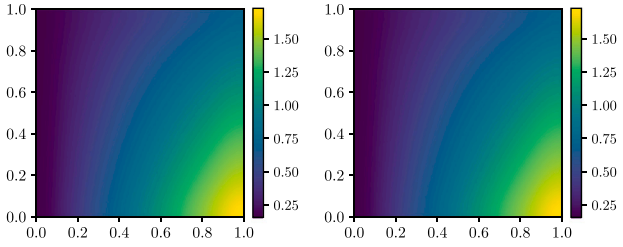


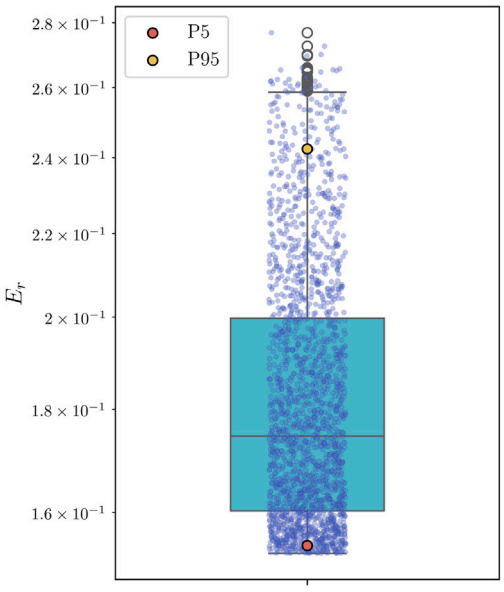
Fig. 22. $K(x, y)$ actual (left) vs. predicted (right) in the heterogeneous problem.



(a) $u(x, y)$ actual (left) vs. predicted (right) in P_5

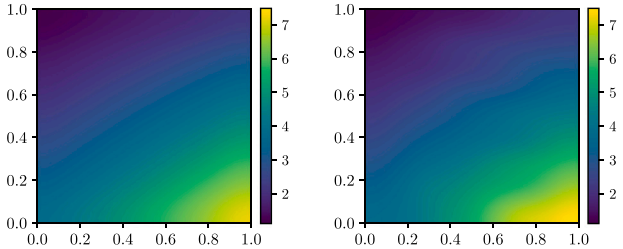


(b) $u(x, y)$ actual (left) vs. predicted (right) in P_{95}

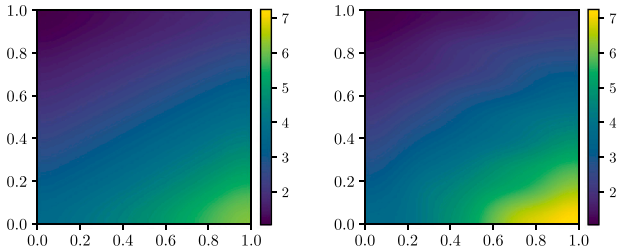


(c) Boxplot of relative error in predictions of $u(x, y)$

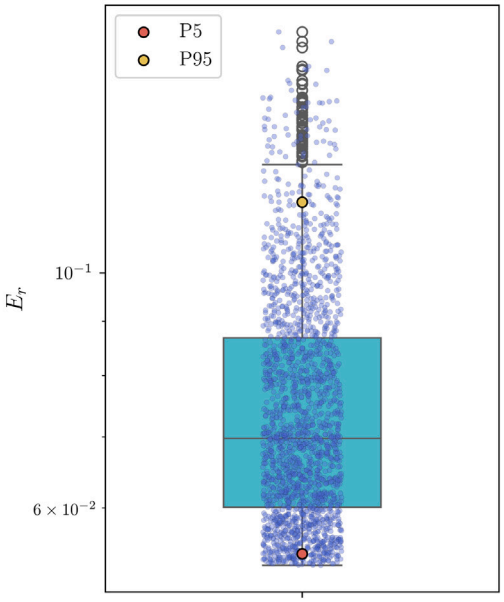
Fig. 23. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for $u(x, y)$ in the anisotropic problem.



(a) $q_x(x, y)$ actual (left) vs. predicted (right) in P_5

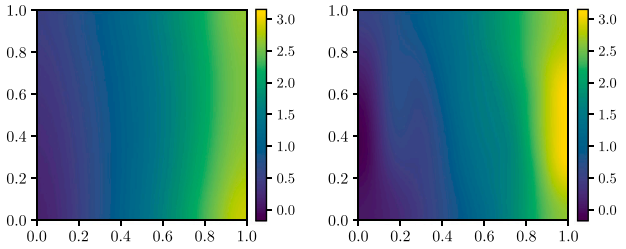


(b) $q_x(x, y)$ actual (left) vs. predicted (right) in P_{95}

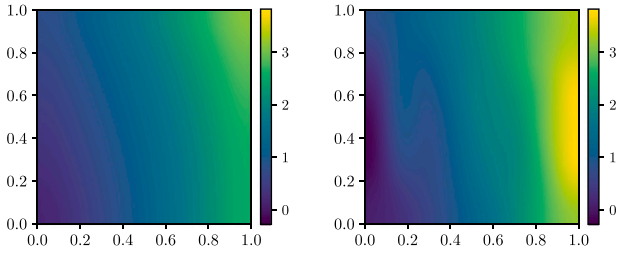


(c) Boxplot of relative error in predictions of $q_x(x, y)$

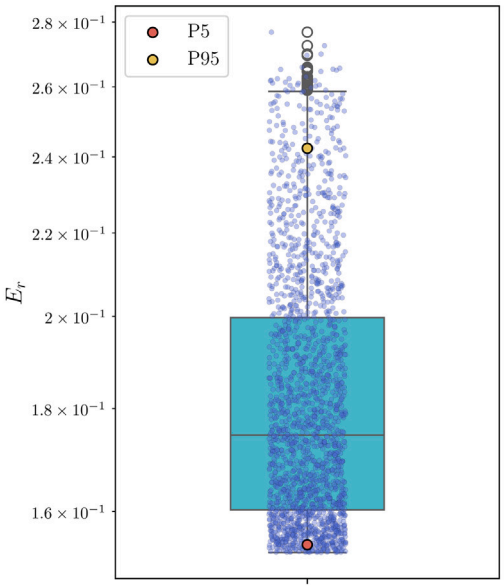
Fig. 24. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for $q_x(x, y)$ in the anisotropic problem.



(a) $q_y(x, y)$ actual (left) vs. predicted (right) in P_5

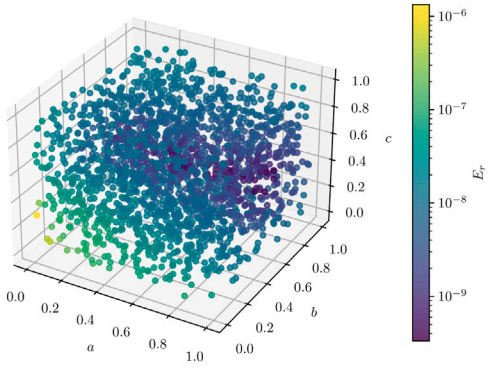


(b) $q_y(x, y)$ actual (left) vs. predicted (right) in P_{95}

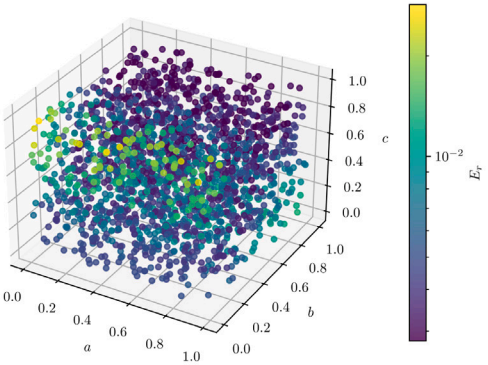


(c) Boxplot of relative error in predictions of $q_y(x, y)$

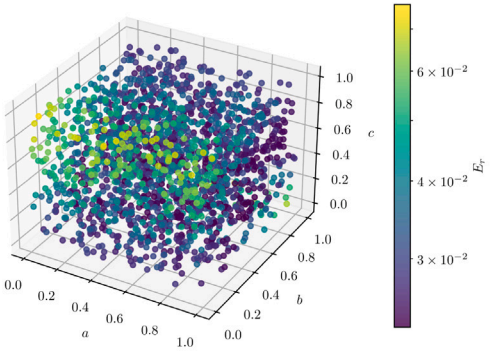
Fig. 25. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for $q_y(x, y)$ in the anisotropic problem.



(a) E_r of predicted solutions $u(x, y)$ for parameters a, b, and c.

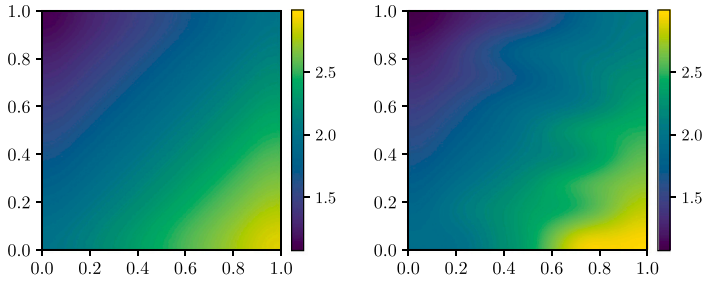


(b) E_r of predicted flux $q_x(x, y)$ for parameters a, b, and c.

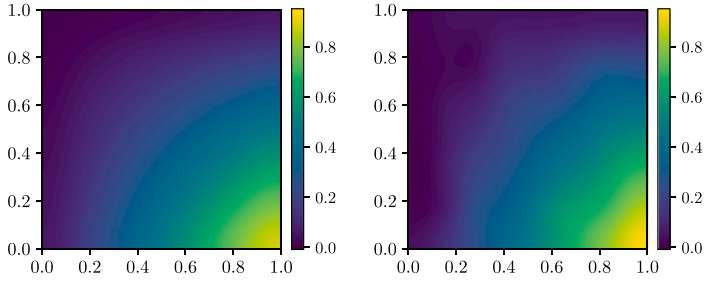


(c) E_r of predicted flux $q_y(x, y)$ for parameters a, b, and c.

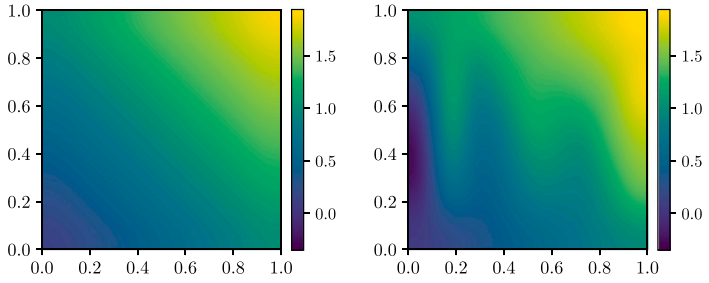
Fig. 26. E_r of predicted variables for parameters a, b, and c in the anisotropic problem.



(a) $K_{xx}(x, y)$ actual (left) vs. predicted (right)



(b) $K_{xy}(x, y) = K_{yx}(x, y)$ actual (left) vs. predicted (right)



(c) $K_{yy}(x, y)$ actual (left) vs. predicted (right)

Fig. 27. Predictions and actual components $K_{ij}(x, y)$ of diffusivity tensor in anisotropic tensor.

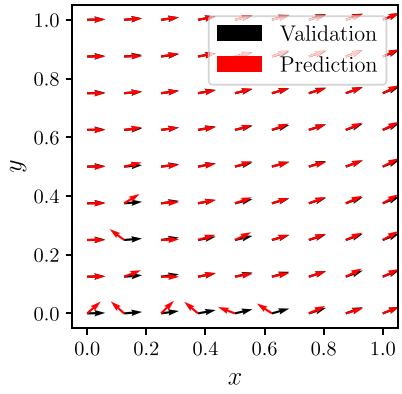
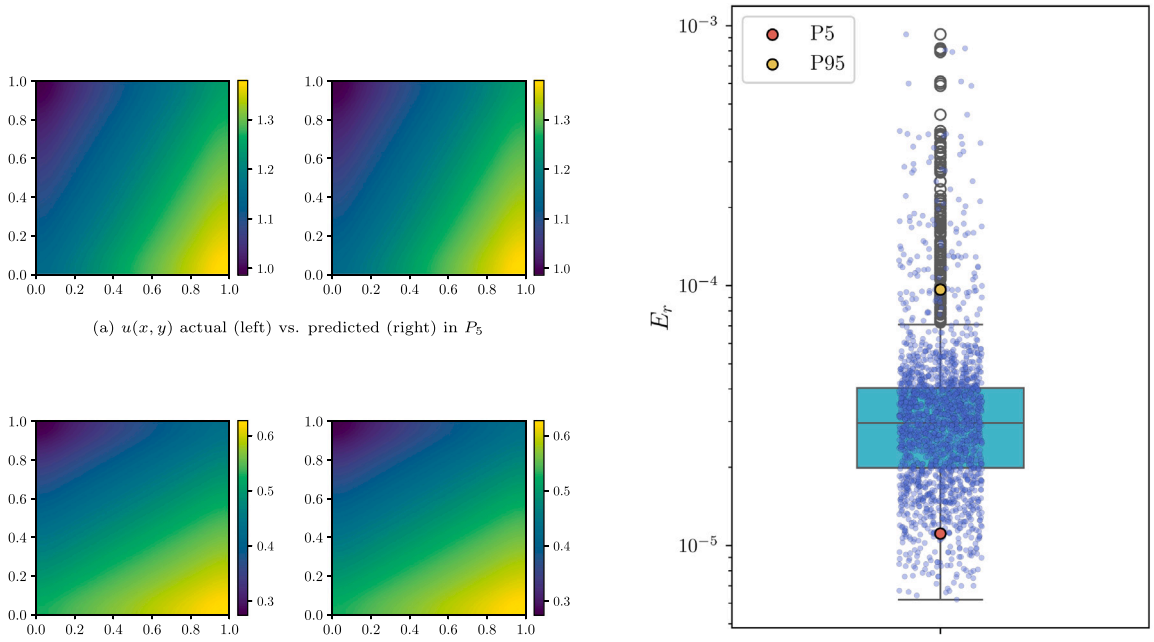


Fig. 28. First principal direction of the diffusion tensor (real vs predicted) in the anisotropic problem.

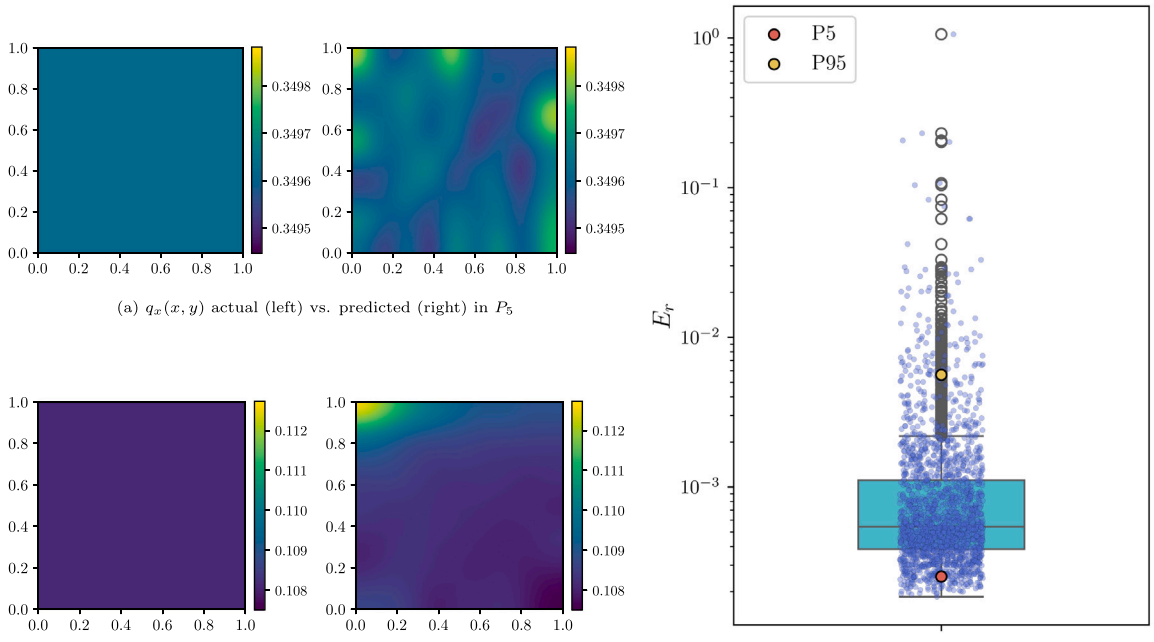


(a) $u(x, y)$ actual (left) vs. predicted (right) in P_5

(b) $u(x, y)$ actual (left) vs. predicted (right) in P_{95}

(c) Boxplot of relative error in predictions of u

Fig. 29. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for $u(x, y)$ in the nonlinear problem NL1.

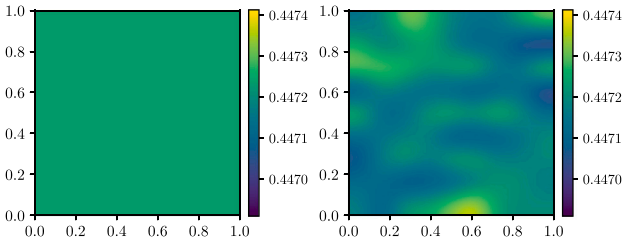


(a) $q_x(x, y)$ actual (left) vs. predicted (right) in P_5

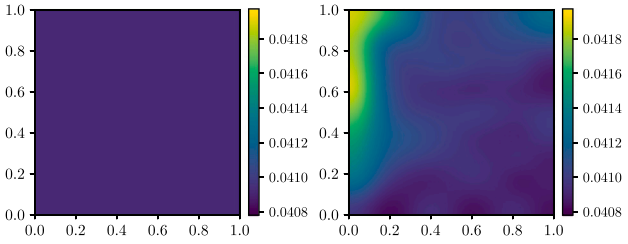
(b) $q_x(x, y)$ actual (left) vs. predicted (right) in P_{95}

(c) Boxplot of relative error in predictions of q_x

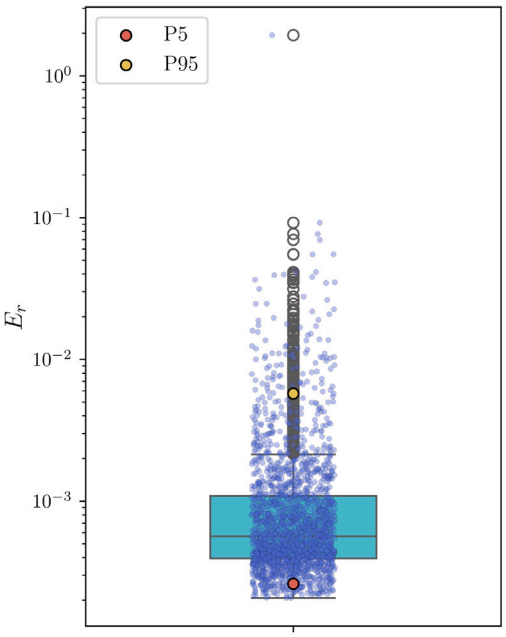
Fig. 30. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for $q_x(x, y)$ in the nonlinear problem NL1.



(a) $q_y(x, y)$ actual (left) vs. predicted (right) in P_5

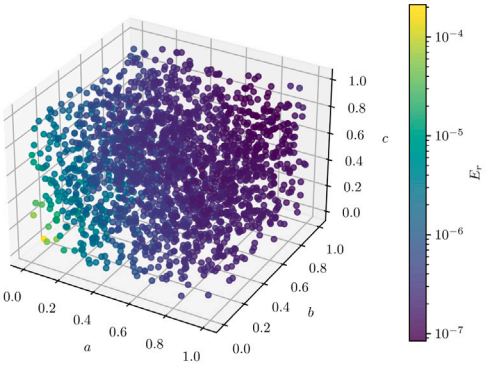


(b) $q_y(x, y)$ actual (left) vs. predicted (right) in P_{95}

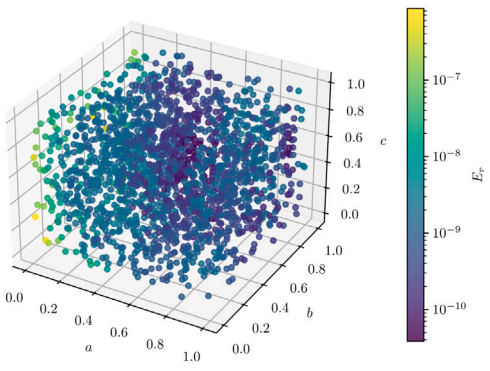


(c) Boxplot of relative error in predictions of $q_y(x, y)$

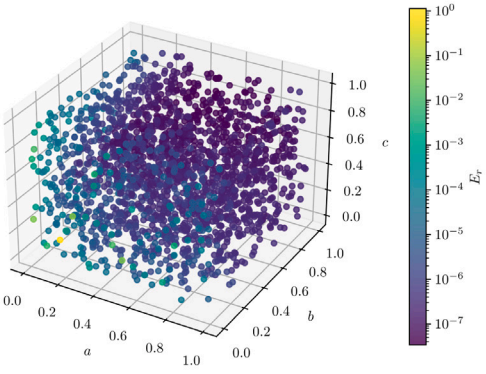
Fig. 31. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for $q_y(x, y)$ in the nonlinear problem NL1.



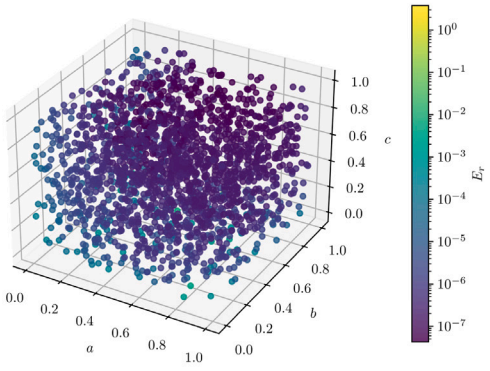
(a) E_r of predicted flux $k(u(x, y))$ for parameters a, b, and c.



(b) E_r of predicted flux $u(x, y)$ for parameters a, b, and c.

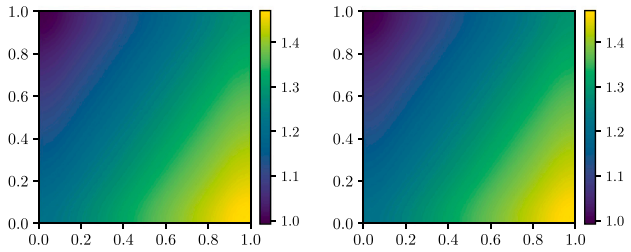


(c) E_r of predicted flux $q_x(x, y)$ for parameters a, b, and c.

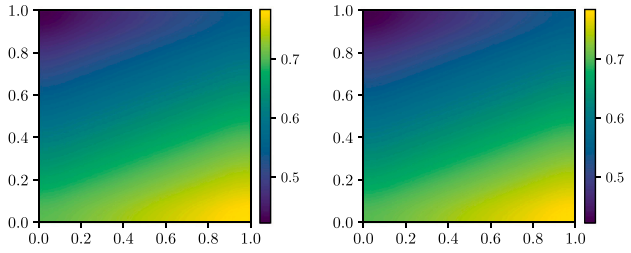


(d) E_r of predicted flux $q_y(x, y)$ for parameters a, b, and c.

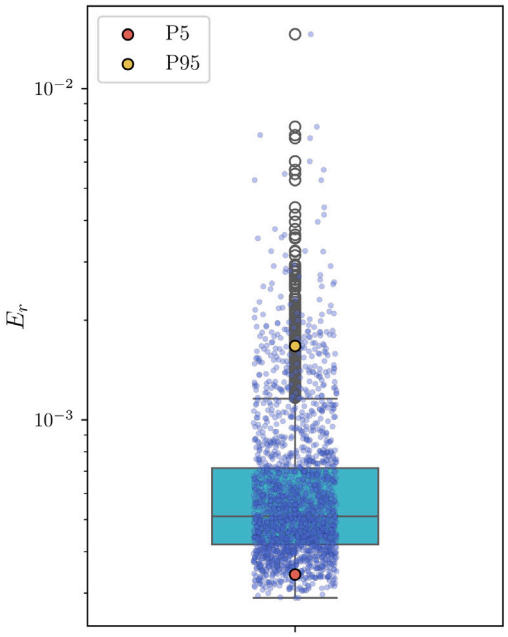
Fig. 32. E_r of predicted variables for parameters a, b, and c in the nonlinear problem NL1.



(a) $k(u(x, y))$ actual (left) vs. predicted (right) in P_5



(b) $k(u(x, y))$ actual (left) vs. predicted (right) in P_{95}



(c) Boxplot of relative error in predictions of K

Fig. 33. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for K in nonlinear the problem NL1.

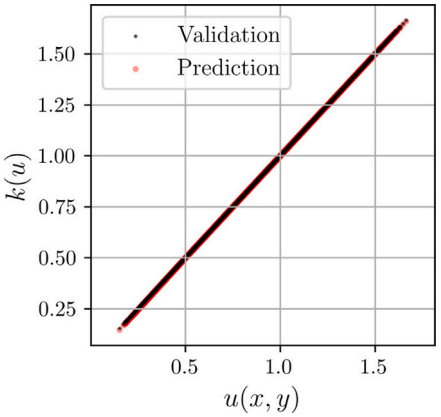
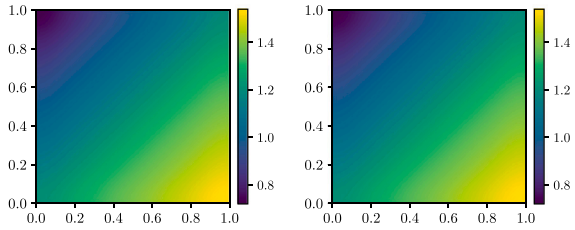
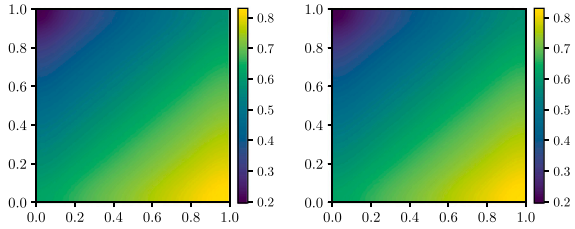


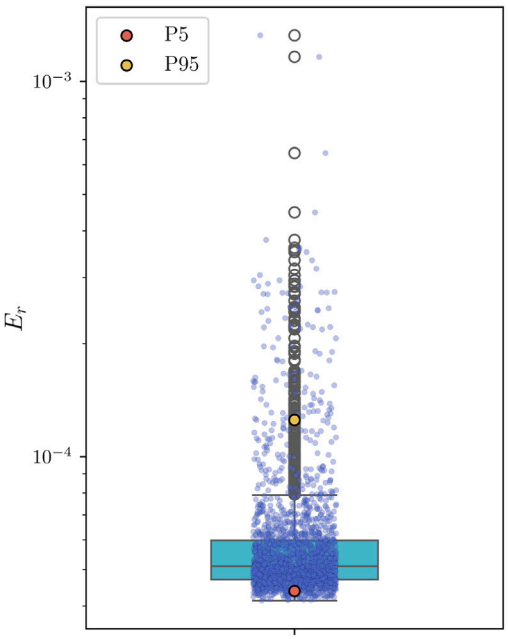
Fig. 34. Comparison between the mean value of $k(u)$ and the mean value of $u(x, y)$ calculated from the entire validation dataset in the nonlinear problem NL1.



(a) $u(x, y)$ actual (left) vs. predicted (right) in P_5

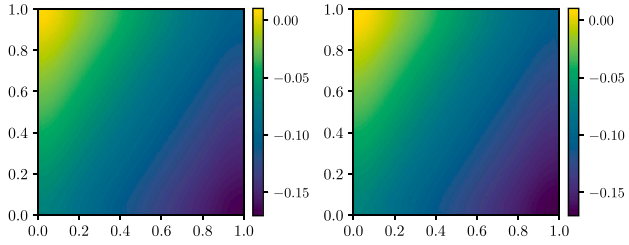


(b) $u(x, y)$ actual (left) vs. predicted (right) in P_{95}

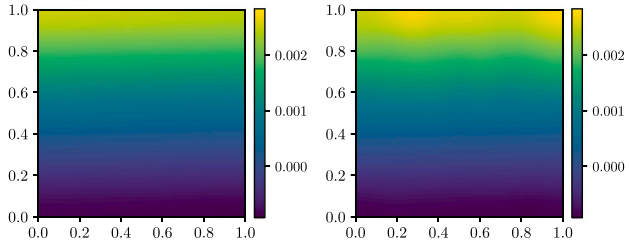


(c) Boxplot of relative error in predictions of u

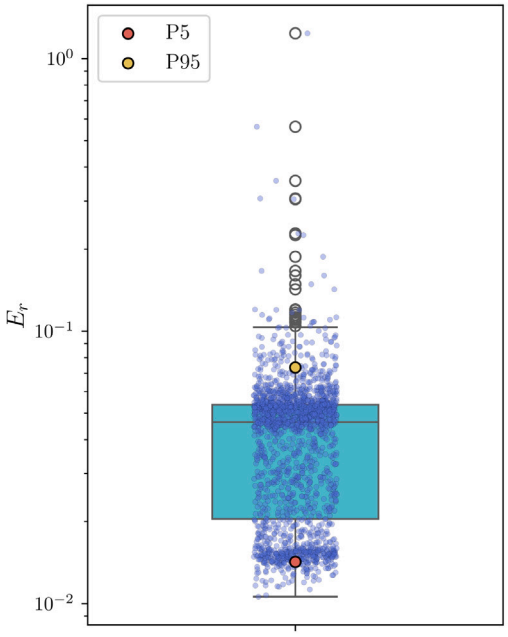
Fig. 35. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for $u(x, y)$ in the nonlinear problem NL2.



(a) $q_x(x, y)$ actual (left) vs. predicted (right) in P_5

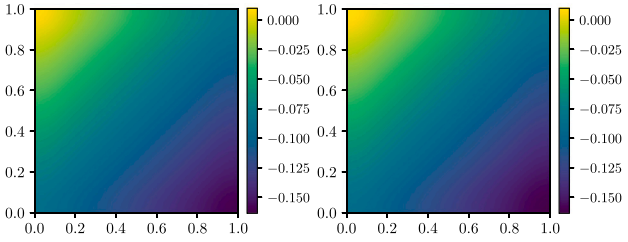


(b) $q_x(x, y)$ actual (left) vs. predicted (right) in P_{95}

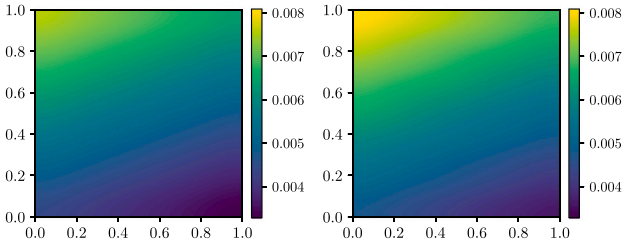


(c) Boxplot of relative error in predictions of q_x

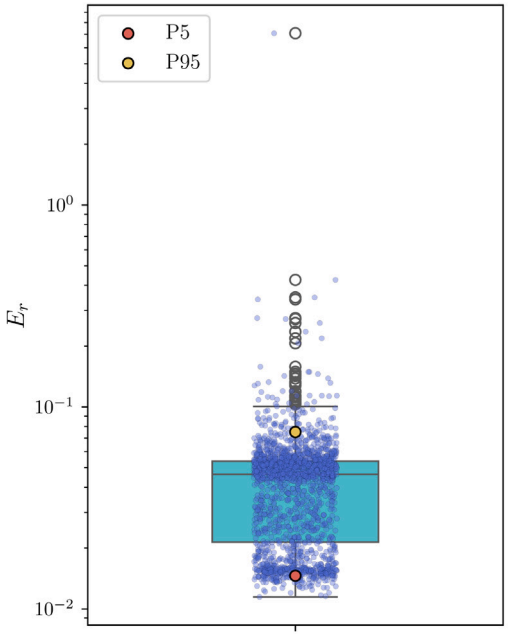
Fig. 36. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for $q_x(x, y)$ in the nonlinear problem NL2.



(a) $q_y(x, y)$ actual (left) vs. predicted (right) in P_5

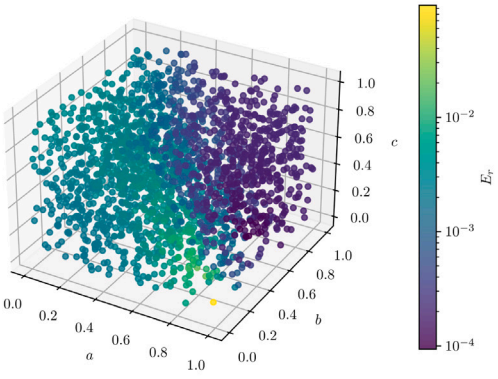


(b) $q_y(x, y)$ actual (left) vs. predicted (right) in P_{95}

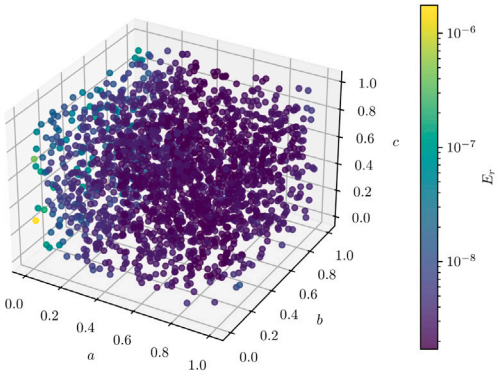


(c) Boxplot of relative error in predictions of $q_y(x, y)$

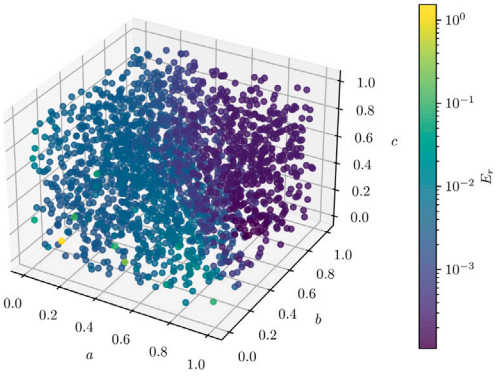
Fig. 37. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for $q_y(x, y)$ in the nonlinear problem NL2.



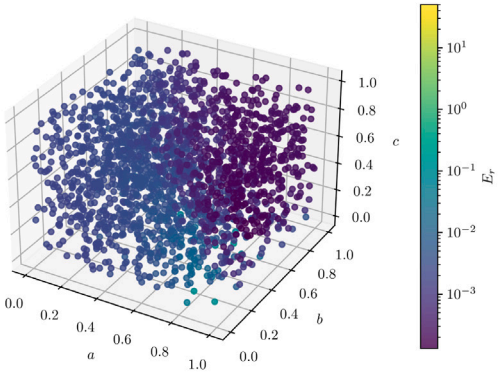
(a) E_r of predicted flux $k(u(x, y))$ for parameters a, b, and c.



(b) E_r of predicted flux $u(x, y)$ for parameters a, b, and c.

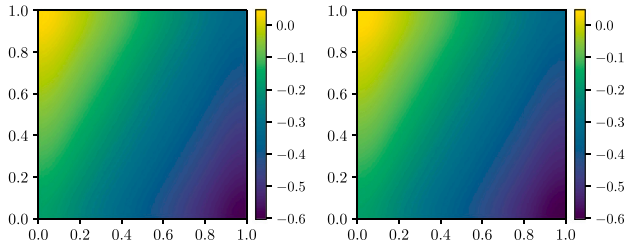


(c) E_r of predicted flux $q_x(x, y)$ for parameters a, b, and c.

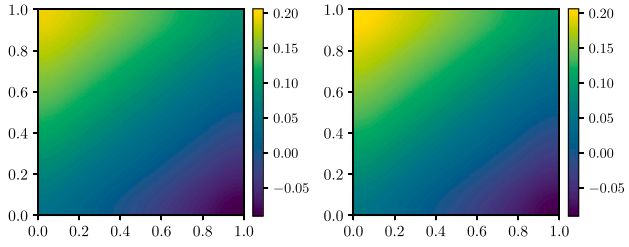


(d) E_r of predicted flux $q_y(x, y)$ for parameters a, b, and c.

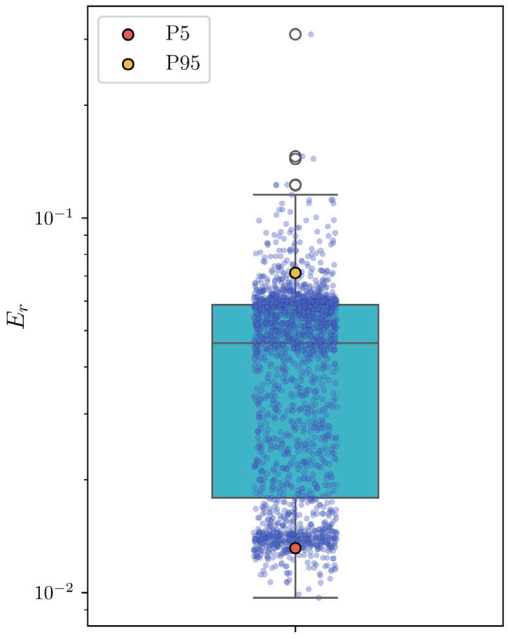
Fig. 38. E_r of predicted variables for parameters a, b, and c in the nonlinear problem NL2.



(a) $k(u(x, y))$ actual (left) vs. predicted (right) in P_5



(b) $k(u(x, y))$ actual (left) vs. predicted (right) in P_{95}



(c) Boxplot of relative error in predictions of K

Fig. 39. Boxplot and two results (of relative error in percentile 5% and percentile 95%) for K in the nonlinear problem NL2.

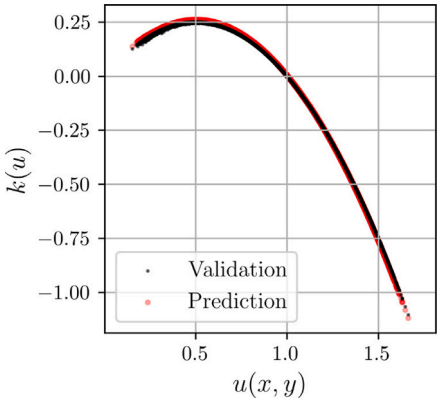


Fig. 40. Comparison between the mean value of $k(u)$ and the mean value of $u(x, y)$ calculated from the entire validation dataset in the nonlinear problem NL2.

References

Abadi, Martín, Agarwal, Ashish, Barham, Paul, Brevdo, Eugene, Chen, Zhifeng, Citro, Craig, Corrado, Greg S, Davis, Andy, Dean, Jeffrey, Devin, Matthieu, et al., 2016. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. arXiv preprint arXiv:1603.04467.

Aneshensel, Carol S., 2013. Theory-Based Data Analysis for the Social Sciences. SAGE Publications, Inc.

Anwar, Syed Muhammad, Majid, Muhammad, Qayyum, Adnan, Awais, Muhammad, Alnowami, Majdi, Khan, Muhammad Khurram, 2018. Medical image analysis using convolutional neural networks: a review. J. Med. Syst. 42 (11), 226.

Atzori, L., Iera, A., Morabito, G., 2010. The internet of things: A survey. Comput. Netw. 54 (15), 2787–2805.

Ayensa-Jiménez, Jacobo, Doweidar, Mohamed H, Sanz-Herrera, Jose A, Doblaré, Manuel, 2018. A new reliability-based data-driven approach for noisy experimental data with physical constraints. Comput. Methods Appl. Mech. Engrg. 328, 752–774.

Ayensa-Jiménez, Jacobo, Doweidar, Mohamed H, Sanz-Herrera, Jose A, Doblaré, Manuel, 2019. An unsupervised data completion method for physically-based data-driven models. Comput. Methods Appl. Mech. Engrg. 344, 120–143.

Ayensa-Jiménez, Jacobo, Doweidar, Mohamed H., Sanz-Herrera, Jose A., Doblaré, Manuel, 2021. Prediction and identification of physical systems by means of physically-guided neural networks with meaningful internal layers. Comput. Methods Appl. Mech. Engrg. 381, 113816.

Ayensa-Jiménez, Jacobo, Doweidar, Mohamed H, Sanz-Herrera, Jose A, Doblare, Manuel, 2022. Understanding glioblastoma invasion using physically-guided neural networks with internal variables. PLoS Comput. Biol. 18 (4), e1010019.

Ayensa-Jiménez, Jacobo, Orera-Echeverría, Javier, Doblare, Manuel, 2024. Predicting and explaining nonlinear material response using deep physically guided neural networks with internal variables. Math. Mech. Solids 10812865241257850.

Bar, Leah, Sochen, Nir A., 2019. Unsupervised deep learning algorithm for PDE-based forward and inverse problems. arXiv. arXiv:1904.05417. URL https://api.semanticscholar.org/CorpusID:119308075.

Barenblatt, Grigory Isaakovich, Entov, Vladimir Mordukhovich, Ryzhik, Viktor Mikhaĭlovich, 1989. Theory of Fluid Flows Through Natural Rocks. Kluwer Academic Publishers, Norwell, MA (USA).

Barles, Guy, Soner, Halil Mete, 1998. Option pricing with transaction costs and a nonlinear Black-Scholes equation. Finance Stoch. 2 (4), 369–397.

Bergstra, James, Bastien, Frédéric, Breuleux, Olivier, Lamblin, Pascal, Pascanu, Razvan, Delalleau, Olivier, Desjardins, Guillaume, Warde-Farley, David, Goodfellow, Ian, Bergeron, Arnaud, et al., 2011. Theano: Deep learning on gpus with python. In: NIPS 2011, BigLearning Workshop, Granada, Spain. 3, pp. 1–48, Citeseer.

Berry, D.M., 2011. The computational turn: Thinking about the digital humanities. Cult. Mach. 12.

Blasch, Erik P, Darema, Frederica, Ravela, Sai, Aved, Alex J, 2022. Handbook of Dynamic Data Driven Applications Systems: Volume 1. Springer Nature.

Bonet, Javier, Gil, Antonio J., Wood, Richard D., 2016. Nonlinear Solid Mechanics for Finite Element Analysis: Statics. Cambridge University Press.

Boyd, John P., 2001. Chebyshev and Fourier Spectral Methods. Courier Corporation.

Bronstein, Michael M, Bruna, Joan, LeCun, Yann, Szlam, Arthur, Vandergheynst, Pierre, 2017. Geometric deep learning: going beyond euclidean data. IEEE Signal Process. Mag. 34 (4), 18–42.

Chai, Chengliang, Jin, Kasisen, Tang, Nan, Fan, Ju, Qiao, Lianpeng, Wang, Yuping, Luo, Yuyu, Yuan, Ye, Wang, Guoren, 2024. Mitigating data scarcity in supervised machine learning through reinforcement learning guided data generation. In: 2024 IEEE 40th International Conference on Data Engineering. ICDE, IEEE, pp. 3613–3626.

Ciaurri, Óscar, Roncal, Luz, Stinga, Pablo Raúl, Torrea, José L, Varona, Juan Luis, 2018. Nonlocal discrete diffusion equations and the fractional discrete Laplacian, regularity and applications. Adv. Math. 330, 688–738.

Cueto, Elias, Chinesta, Francisco, 2023. Thermodynamics of learning physical phenomena. Arch. Comput. Methods Eng. 30 (8), 4653–4666.

Cuomo, Salvatore, Di Cola, Vincenzo Schiano, Giampaolo, Fabio, Rozza, Gianluigi, Raissi, Maziar, Piccialli, Francesco, 2022. Scientific machine learning through physics–informed neural networks: Where we are and what's next. J. Sci. Comput. 92 (3), 88.

Cybenko, George, 1989. Approximations by superpositions of a sigmoidal function. Math. Control Signals Systems 2, 183–192.

Darema, Frederica, 2004. Dynamic data driven applications systems: A new paradigm for application simulations and measurements. In: International Conference on Computational Science. Springer, pp. 662–669.

David, Marco, Méhats, Florian, 2023. Symplectic learning for Hamiltonian neural networks. J. Comput. Phys. 494, 112495.

Fernández, Mauricio, Jamshidian, Mostafa, Böhlke, Thomas, Kersting, Kristian, Weeger, Oliver, 2021. Anisotropic hyperelastic constitutive models for finite deformations combining material theory and data-driven approaches with application to cubic lattice metamaterials. Comput. Mech. 67, 653–677.

Fischer, Paul, Mullen, Julia, et al., 2001. Filter-based stabilization of spectral element methods. C. R. de L'Acad. Des Sci. Ser. I Math. 332 (3), 265–270.

Frank, Till Daniel, 2005. Nonlinear Fokker-Planck Equations: Fundamentals and Applications. Springer Science & Business Media.

Garanger, Kévin, Kraus, Julie, Rimoli, Julian J., 2024. Symmetry-enforcing neural networks with applications to constitutive modeling. Extrem. Mech. Lett. 71, 102188.

Gingold, Robert A., Monaghan, Joseph J., 1977. Smoothed particle hydrodynamics: theory and application to non-spherical stars. Mon. Not. R. Astron. Soc. 181 (3), 375–389.

Glorot, Xavier, Bengio, Yoshua, 2010. Understanding the difficulty of training deep feedforward neural networks. In: Teh, Yee Whye, Titterington, Mike (Eds.), Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics. In: Proceedings of Machine Learning Research, 9, PMLR, Chia Laguna Resort, Sardinia, Italy, pp. 249–256.

Gould, P., 1981. Letting the data speak for themselves. Ann. Assoc. Am. Geogr. 71 (2), 166–176.

Greydanus, Samuel, Dzamba, Misko, Yosinski, Jason, 2019. Hamiltonian neural networks. In: Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché Buc, F., Fox, E., Garnett, R. (Eds.), Advances in Neural Information Processing Systems. 32, Curran Associates, Inc.

Gulli, Antonio, Pal, Sujit, 2017. Deep Learning with Keras. Packt Publishing Ltd.

Haghighat, Ehsan, Raissi, Maziar, Moure, Adrian, Gomez, Hector, Juanes, Ruben, 2021. A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics. Comput. Methods Appl. Mech. Engrg. 379, 113741.

Heider, Yousef, Wang, Kun, Sun, WaiChing, 2020. SO (3)-invariance of informed-graph-based deep neural network for anisotropic elastoplastic materials. Comput. Methods Appl. Mech. Engrg. 363, 112875.

Hoffmann, Frank, Bertram, Torsten, Mikut, Ralf, Reischl, Markus, Nelles, Oliver, 2019. Benchmarking in classification and regression. Wiley Interdiscip. Rev.: Data Min. Knowl. Discov. 9 (5), e1318.

Hornik, Kurt, 1991. Approximation capabilities of multilayer feedforward networks. Neural Netw. 4 (2), 251–257.

Jin, Pengzhan, Zhang, Zhen, Zhu, Aiqing, Tang, Yifa, Karniadakis, George Em, 2020. SympNets: Intrinsic structure-preserving symplectic networks for identifying Hamiltonian systems. Neural Netw. 132, 166–179.

Karniadakis, George Em, Kevrekidis, Ioannis G, Lu, Lu, Perdikaris, Paris, Wang, Sifan, Yang, Liu, 2021. Physics-informed machine learning. Nat. Rev. Phys. 3 (6), 422–440.

Karpatne, Anuj, Atluri, Gowtham, Faghmous, James H, Steinbach, Michael, Banerjee, Arindam, Ganguly, Auroop, Shekhar, Shashi, Samatova, Nagiza, Kumar, Vipin, 2017a. Theory-guided data science: A new paradigm for scientific discovery from data. IEEE Trans. Knowl. Data Eng. 29 (10), 2318–2331.

Karpatne, Anuj, Watkins, William, Read, Jordan S., Kumar, Vipin, 2017b. Physics-guided neural networks (PGNN): An application in lake temperature modeling. arXiv. arXiv:1710.11431.

Kettinger, William J., Li, Yuan, 2010. The infological equation extended: towards conceptual clarity in the relationship between data, information and knowledge. Eur. J. Inf. Syst. 19 (4), 409–421.

Kingma, Diederik P., Ba, Jimmy, 2014. Adam: A method for stochastic optimization. CoRR. arXiv:1412.6980.

Kitchin, R., 2013. Big data and human geography: Opportunities, challenges and risks. Dialogues Hum. Geogr. 3, 262–267.

Kitchin, Rob, 2014. Big data, new epistemologies and paradigm shifts. Big Data Soc. 1 (1), 2053951714528481.

Krizhevsky, Alex, Sutskever, Ilya, Hinton, Geoffrey E., 2012. ImageNet classification with deep convolutional neural networks. In: Proceeding NIPS'12 Proceedings of the 25th International Conference on Neural Information Processing Systems. pp. 1097–1105.

Kutz, J Nathan, Brunton, Steven L, Brunton, Bingni W, Proctor, Joshua L, 2016. Dynamic Mode Decomposition: Data-Driven Modeling of Complex Systems. SIAM.

Langtangen, Hans Petter, 1999. Computational Partial Differential Equations: Numerical Methods and Diffpack Programming. 2, Springer, Berlin.

Larsson, S., Thomee, V., 2009. Partial Differential Equations with Numerical Methods. Springer Verlag, Belin-Heidelberg.

LeCun, Y.C., 2019. Deep learning hardware: Past, present, and future. In: Proceedings 2019 IEEE International Solid- State Circuits Conference - (ISSCC), San Francisco, CA, USA. pp. 12–19.

Li, Xiang, Yan, Ziming, Liu, Zhanli, 2019. Combination and application of machine learning and computational mechanics. Chin. Sci. Bull. 64 (7), 635–648.

Ling, Julia, Jones, Reese, Templeton, Jeremy, 2016. Machine learning strategies for systems with invariance properties. J. Comput. Phys. 318, 22–35.

Long, Zichao, Lu, Yiping, Dong, Bin, 2019. PDE-Net 2.0: Learning PDEs from data with a numeric-symbolic hybrid deep network. J. Comput. Phys. 399, 108925.

Lopez-Moreno, Ignacio, Gonzalez-Dominguez, Javier, Plchot, Oldrich, Martinez, David, Gonzalez-Rodriguez, Joaquin, Moreno, Pedro, 2014. Automatic language identification using deep neural networks. In: Proceeding ICASSP, 2014, Proceeding of the IEEE International Conference on Acoustic, Speech and Signal Processing. pp. 5337–5341.

Lu, Lu, Meng, Xuhui, Mao, Zhiping, Karniadakis, George Em, 2021. DeepXDE: A deep learning library for solving differential equations. SIAM Rev. 63 (1), 208–228.

Lu, Zhou, Pu, Hongming, Wang, Feicheng, Hu, Zhiqiang, Wang, Liwei, 2017. The expressive power of neural networks: A view from the width. In: Advances in Neural Information Processing Systems. pp. 6231–6239.

Maggiora, Gerald M., Elrod, David W., Trenary, Robert G., 1992. Computational neural networks as model-free mapping devices. J. Chem. Inf. Comput. Sci. 32 (6), 732–741.

Maharana, Kiran, Mondal, Surajit, Nemade, Bhushankumar, 2022. A review: Data pre-processing and data augmentation techniques. Glob. Trans. Proc. 3 (1), 91–99.

Manyika, J., Chui, M., Brown, B., Bughin, J., Dobbs, R., Roxburgh, C., Hung-Byers, A., 2011. Big Data: The Next Frontier for Innovation, Competition, and Productivity. McKinsey Global Institute Reports.

Masi, Filippo, Stefanou, Ioannis, 2022. Multiscale modeling of inelastic materials with thermodynamics-based artificial neural networks (TANN). Comput. Methods Appl. Mech. Engrg. 398, 115190.

Masi, Filippo, Stefanou, Ioannis, Vannucci, Paolo, Maffi-Berthier, Victor, 2021. Thermodynamics-based artificial neural networks for constitutive modeling. J. Mech. Phys. Solids 147, 104277.

McCann, Michael T., Jin, Kyong Hwan, Unser, Michael, 2017. Convolutional neural networks for inverse problems in imaging: A review. IEEE Signal Process. Mag. 34 (6), 85–95.

Minto, W., 2018. Logic, Inductive and Deductive, Alpha ed..

Nayroles, B., Touzot, G., Villon, P., 1992. Generalizing the finite element method: diffuse approximation and diffuse elements. Comput. Mech. 10 (5), 307–318.

Osserman, Robert, 2013. A survey of minimal surfaces. Courier Corporation.

Paszke, Adam, Gross, Sam, Massa, Francisco, Lerer, Adam, Bradbury, James, Chanan, Gregory, Killeen, Trevor, Lin, Zeming, Gimelshein, Natalia, Antiga, Luca, et al., 2019. Pytorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems. pp. 8026–8037.

Peherstorfer, Benjamin, Willcox, Karen, 2015. Dynamic data-driven reduced-order models. Comput. Methods Appl. Mech. Engrg. 291, 21–41.

Purwins, Hendrik, Li, Bo, Virtanen, Tuomas, Schlüter, Jan, Chang, Shuo-Yiin, Sainath, Tara, 2019. Deep learning for audio signal processing. IEEE J. Sel. Top. Signal Process. 13 (2), 206–219.

Raghupathi, Wullianallur, Viju, Raghupathi, 2014. Network-based marketing: Identifying likely adopters via consumer networks. Heal. Inf. Sci. Syst. 2, 3–7.

Raissi, Maziar, Perdikaris, Paris, Karniadakis, George Em, 2017. Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations. arXiv preprint arXiv:1711.10561.

Raissi, Maziar, Perdikaris, Paris, Karniadakis, George E., 2019. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J. Comput. Phys. 378, 686–707.

Rawat, Waseem, Wang, Zenghui, 2017. Deep convolutional neural networks for image classification: A comprehensive review. Neural Comput. 29 (9), 2352–2449.

Roache, Patrick J., 2001. Code verification by the method of manufactured solutions. J. Fluids Eng. 124 (1), 4–10.

Ros-Oton, Xavier, 2015. Nonlocal elliptic equations in bounded domains: a survey. arXiv preprint arXiv:1504.04099.

Ruas, V., 2016. Numerical Methods for Partial Differential Equations: An Introduction. John Wiley and Sons Ltd, Chichester, West Sussex, United Kingdom.

Stulp, Freek, Sigaud, Olivier, 2015. Many regression algorithms, one unified model: A review. Neural Netw. 69, 60–79.

Sukumar, Natarajan, Moran, Brian, Belytschko, Ted, 1998. The natural element method in solid mechanics. Internat. J. Numer. Methods Engrg. 43 (5), 839–887.

Xu, Kailai, Huang, Daniel Z., Darve, Eric, 2021. Learning constitutive relations using symmetric positive definite neural networks. J. Comput. Phys. 428, 110072.

Xu, Feiyu, Uszkoreit, Hans, Du, Yangzhou, Fan, Wei, Zhao, Dongyan, Zhu, Jun, 2019. Explainable AI: A brief survey on history, research areas, approaches and challenges. In: CCF International Conference on Natural Language Processing and Chinese Computing. Springer, pp. 563–574.

Xue, Ying, 2019. An overview of overfitting and its solutions. J. Phys. Conf. Ser. 1168, 022022.

Yoo, Hyeon-Joong, 2015. Deep convolution neural networks in computer vision: a review. IEIE Trans. Smart Process. Comput. 4 (1), 35–43.

Zienkiewicz, Olgierd Cecil, Morice, P.B., 1971. The Finite Element Method in Engineering Science. 1977, McGraw-Hill, London.